control-systems: the Control Systems program, user guide

control-systems-ai999x 1.6.0, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Control Systems covers transfer functions, poles and zeros, step and frequency response, stability (Routh-Hurwitz, gain and phase margins, Nyquist), root locus, state space, controllability and observability, pole placement, LQR, Kalman filters, discretization and PID tuning. This guide is about `control-systems`, the model as a self-contained program for Windows and Linux: 97 capabilities in 16 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/control-systems-ai999x/); the Python package (`pip install control-systems-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.6.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://downloads.ai999x.com/desktop/1.6.0/SHA256SUMS; the newest version is always under https://downloads.ai999x.com/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Control Systems	Windows x86-64	https://downloads.ai999x.com/desktop/1.6.0/ai999x-control-systems-1.6.0-windows-x86_64.zip
Desktop app: Control Systems	Linux x86-64	https://downloads.ai999x.com/desktop/1.6.0/ai999x-control-systems-1.6.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://downloads.ai999x.com/desktop/1.6.0/ai999x-1.6.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://downloads.ai999x.com/desktop/1.6.0/ai999x-1.6.0-linux-x86_64.tar.gz
Command line	Windows x86-64	https://downloads.ai999x.com/desktop/1.6.0/ai999x-cli-1.6.0-windows-x86_64.zip
Command line	Linux x86-64	https://downloads.ai999x.com/desktop/1.6.0/ai999x-cli-1.6.0-linux-x86_64.tar.gz

Desktop app

Control Systems in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-control-systems-1.6.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-control-systems.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.6.0/ai999x-control-systems-1.6.0-linux-
x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.6.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-control-systems-1.6.0-linux-x86_64.tar.gz
./ai999x-control-systems-1.6.0-linux-x86_64/ai999x-control-systems
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.6.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.6.0/ai999x-1.6.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.6.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.6.0-linux-x86_64.tar.gz
./ai999x-1.6.0-linux-x86_64/ai999x
```

Command line, Linux

The bundle holds one program per model; `control-systems` is this one. The checksum line should print OK.

```
curl -LO https://downloads.ai999x.com/desktop/1.6.0/ai999x-cli-1.6.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.6.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.6.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.6.0-linux-x86_64:$PATH"' >> ~/.bashrc &&
source ~/.bashrc
control-systems --version
```

Command line, Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.6.0-windows-x86_64.zip"
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.6.0/$F" -OutFile $F
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.6.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F
SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.6.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path",
"User") + ";$D", "User")
control-systems --version
```

In your browser

Open https://www.ai999x.com/models/control-systems-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install control-systems-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ control-systems --version
control-systems-ai999x 1.6.0 (Rust build, ax-engine 1.6.0)
```

2.2 Ask in words

Write the question as you would say it: the program reads the quantities and their units, picks the capability and answers.

```
$ control-systems ask "stability margins: num = [10], den = [1, 3, 2, 0]"
Read as: Gain and phase margins
gm = 0.6
gm_db = -4.436974992
w_pc = 1.414213562 rad/s
pm_deg = -12.99720802
w_gc = 1.802203305 rad/s
  by: Gain and phase margins  (crossovers as positive real roots of |N|^2 - |D|^2 and Im(N(jw)
conj D(jw)), Newton-polished)
  assumes: exactly one gain crossover and one phase crossover
```

2.3 Find a capability

Search the capabilities by what you want to know; the first column is the id to pass to info and solve.

```
$ control-systems search "gain and phase margins"
ctrl_stability_margins              Gain and phase margins  (stability / margins)
ctrl_sensitivity_peak               Peak sensitivity Ms and modulus margin  (stability /
robustness)
ctrl_lead_max_phase                 Maximum phase lead of a lead compensator  (compensators /
lead)
ctrl_bode_point                     Bode magnitude and phase at one frequency  (frequency
response / Bode)
ctrl_lead_design                    Lead compensator design for a required phase lead
(compensators / lead)
ctrl_lag_design                     Lag compensator placement  (compensators / lag)
ctrl_pole_stability                 Poles and stability of a characteristic polynomial
(stability / poles)
ctrl_poles_zeros                    Poles, zeros and gain of a transfer function  (transfer
functions / poles and zeros)
… 12 more
```

2.4 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ control-systems list
ctrl_ackermann                          Pole placement by Ackermann's formula
ctrl_arrival_angle                      Root-locus angle of arrival at a complex zero
ctrl_backward_euler                     Backward Euler (backward difference) discretization
ctrl_bandwidth                          -3 dB bandwidth of a transfer function
ctrl_bandwidth_second                   Bandwidth of a standard second-order system
ctrl_bode_point                         Bode magnitude and phase at one frequency
ctrl_breakaway_points                   Root-locus breakaway and break-in points
ctrl_c2d_foh                            First-order-hold (triangle-hold) discretization
… 89 more
```

2.5 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ control-systems info ctrl_stability_margins
Gain and phase margins
  crossovers as positive real roots of |N|^2 - |D|^2 and Im(N(jw) conj D(jw)), Newton-polished
  in   num      numerator of the open loop L(s) (vector) []
  in   den      denominator of L(s) (vector) []
  out  gm       gain margin (ratio) []
  out  gm_db    gain margin in dB []
  out  w_pc     phase-crossover frequency [rad/s]
  out  pm_deg   phase margin in degrees []
  out  w_gc     gain-crossover frequency [rad/s]
```

2.6 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ control-systems solve ctrl_stability_margins "num=[10]" "den=[1, 3, 2, 0]"
gm = 0.6
gm_db = -4.436974992
w_pc = 1.414213562 rad/s
pm_deg = -12.99720802
w_gc = 1.802203305 rad/s
  by: Gain and phase margins  (crossovers as positive real roots of |N|^2 - |D|^2 and Im(N(jw)
conj D(jw)), Newton-polished)
  assumes: exactly one gain crossover and one phase crossover
```

2.7 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ control-systems solve ctrl_stability_margins "num=[10]" "den=[1, 3, 2, 0]" --json
{"outcome":"SOLVED","message":"","value":-12.9972080155,"unit":"","solved_for":"pm_deg","values":
[["gm",0.6,""],["gm_db",-4.43697499233,""],["w_pc",1.41421356237,"rad/s"],
["pm_deg",-12.9972080155,""],["w_gc",1.80220330461,"rad/s"]],"target":"pm_deg","phrase":"gain
margin (ratio)","assumptions":["exactly one gain crossover and one phase
crossover"],"name":"Gain and phase margins","equation":"crossovers as positive real roots of
|N|^2 - |D|^2 and Im(N(jw) conj D(jw)), Newton-polished"}
```

2.8 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ control-systems solve ctrl_stability_margins "num=[10]"
NEEDS_INPUT: ctrl_stability_margins needs den (denominator of L(s)). Example:
solve('ctrl_stability_margins', num=[10], den=[1, 3, 2, 0]).
```

2.9 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | control-systems batch

`cases.jsonl`:

```
{"key": "ctrl_stability_margins", "inputs": {"num": "[10]", "den": "[1, 3, 2, 0]"}}
{"key": "ctrl_routh_hurwitz", "inputs": {"coefficients": "[1, 2, 3, 4, 5]"}}
```

```
$ control-systems batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":-12.9972080155,"unit":"","solved_for":"pm_deg","values":
[["gm",0.6,""],["gm_db",-4.43697499233,""],["w_pc",1.41421356237,"rad/s"],
["pm_deg",-12.9972080155,""],["w_gc",1.80220330461,"rad/s"]],"target":"pm_deg","phrase":"gain
margin (ratio)","assumptions":["exactly one gain crossover and one phase
crossover"],"name":"Gain and phase margins","equation":"crossovers as positive real roots of
|N|^2 - |D|^2 and Im(N(jw) conj D(jw)), Newton-polished"}
{"outcome":"SOLVED","message":"","value":2.0,"unit":"","solved_for":"rhp_roots","values":
[["first_column",[1.0,2.0,1.0,-6.0,5.0],""],["rhp_roots",2,""],
["stable",false,""]],"target":"rhp_roots","phrase":"first column of the Routh
array","assumptions":["no zero in the first column (special cases are refused)"],"name":"Routh-
Hurwitz stability test","equation":"Routh array in exact fractions; sign changes of the first
column"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `control-systems info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]" or "v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`control-systems solve ... --json` prints one answer as one line of JSON. `control-systems batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | control-systems batch`. `control-systems catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]
assumptions	the conditions the answer relies on

answer field	meaning
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["control-systems", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`control-systems figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `control_systems_ai999x.wasm` (https://www.ai999x.com/compute/rust/control_systems_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("control_systems_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `control-systems info <id>` shows the same inputs and outputs in the terminal.

Compensators (3)

Lag compensator placement, `ctrl_lag_design`

C(s) = (s + z)/(s + p) with z = w_c/ratio (default a decade below the crossover) and p = z/beta: zero, pole, DC gain beta, and the phase lag and gain it adds at w_c.

$$C(s) = \frac{s + z}{s + z/\beta}, \quad z = \omega_c/10$$

symbol		meaning	SI unit
`beta`	in	required low-frequency gain increase (> 1)	,
`w_c`	in	gain-crossover frequency in rad/s	,

symbol		meaning	SI unit
`ratio`	in	w_c / zero (default 10) (optional)	,
`zero`	out	zero (rad/s)	,
`pole`	out	pole (rad/s)	,
`dc_gain`	out	DC gain z/p = beta	,
`phase_deg`	out	phase added at w_c in degrees (negative)	,
`magnitude_db`	out	gain added at w_c in dB	,

Applies when: beta > 1; ratio >= 2.

```
$ control-systems solve ctrl_lag_design beta=10 w_c=1
zero = 0.1
pole = 0.01
dc_gain = 10
phase_deg = -5.13765444
magnitude_db = 0.04277946506
  by: Lag compensator placement  (zero a decade below crossover, pole beta times lower)
  beta = 10
  w_c = 1
  assumes: beta > 1; ratio >= 2
```

What the worked example means: This calculation determines the parameters for a lag compensator designed to meet a specific low-frequency gain requirement while maintaining stability. The zero is placed at 0.1, which is typically a decade below the crossover frequency, and the pole is set at 0.01. This configuration results in a DC gain of 10, effectively boosting the system's steady-state accuracy without significantly altering the transient response near the crossover frequency.

Lead compensator design for a required phase lead, `ctrl_lead_design`

alpha = (1 - sin phi)/(1 + sin phi) and T = 1/(w_m sqrt(alpha)) so that C(s) = (T s + 1)/(alpha T s + 1) adds the phase lead phi at w_m; its zero, pole and gain at w_m.

$$\alpha = \frac{1 - \sin\phi_m}{1 + \sin\phi_m}, \quad T = \frac{1}{\omega_m \sqrt{\alpha}}$$

symbol		meaning	SI unit
`phi_max_deg`	in	required maximum phase lead in degrees (0 to 90)	,
`w_m`	in	frequency of the maximum lead in rad/s	,
`alpha`	out	alpha (pole/zero ratio z/p)	,
`T`	out	time constant T	s
`zero`	out	zero corner 1/T (rad/s)	,
`pole`	out	pole corner 1/(alpha T) (rad/s)	,
`gain_db`	out	|C(j w_m)| in dB = -10 log10 alpha	,

Applies when: 0 < phi < 90 degrees; w_m > 0.

```
$ control-systems solve ctrl_lead_design phi_max_deg=30 w_m=1
alpha = 0.3333333333
T = 1.732050808 s
zero = 0.5773502692
pole = 1.732050808
gain_db = 4.771212547
  by: Lead compensator design for a required phase lead  (sin phi_max = (1 - alpha)/(1 +
alpha), w_m = 1/(T sqrt(alpha)))
  phi_max_deg = 30
  w_m = 1
  assumes: 0 < phi < 90 degrees; w_m > 0
```

What the worked example means: This calculation determines the parameters for a lead compensator to achieve a specific phase boost. Using the required maximum phase lead of 30 degrees and the target frequency of 1 rad/s, it computes the ratio alpha as 0.333333 and the time constant T as 1.73205 s seconds. These values define the compensator transfer function, ensuring that the maximum phase contribution occurs exactly at the desired frequency, with a gain increase of 4.77121 dB at that point.

Maximum phase lead of a lead compensator, `ctrl_lead_max_phase`

For C(s) = (s + z)/(s + p) with z < p: the maximum phase lead asin((p - z)/(p + z)), the frequency sqrt(z p) where it occurs, alpha = z/p and the gain there, 10 log10(z/p) dB.

$$\sin\phi_{max} = \frac{p - z}{p + z} = \frac{1 - \alpha}{1 + \alpha}, \quad \omega_m = \sqrt{zp}$$

symbol		meaning	SI unit
`z`	in	zero corner frequency (rad/s)	,
`p`	in	pole corner frequency (rad/s), p > z	,
`phi_max_deg`	out	maximum phase lead in degrees	,
`w_max`	out	frequency of the maximum	rad/s
`alpha`	out	alpha = z/p	,
`magnitude_db`	out	|C(j w_max)| in dB	,

Applies when: 0 < z < p.

```
$ control-systems solve ctrl_lead_max_phase z=1 p=9
phi_max_deg = 53.13010235
w_max = 3 rad/s
alpha = 0.1111111111
magnitude_db = -9.542425094
  by: Maximum phase lead of a lead compensator  (stationary point of the phase)
  z = 1
  p = 9
  assumes: 0 < z < p
```

What the worked example means: This calculation determines the peak phase contribution of a lead compensator defined by zero 1 and pole 9. The result shows that the maximum phase lead reaches 53.1301, which occurs at the geometric mean frequency of 3 rad/s rad/s. At this specific frequency, the compensator provides a gain boost of -9.54243, corresponding to an alpha ratio of 0.111111.

Design (1)

Desired closed-loop poles from overshoot and settling time, `ctrl_specs_to_poles`

zeta from the overshoot, sigma = 4/Ts from the 2 % settling time, wn = sigma/zeta, the poles -sigma +/- j wd and the desired characteristic polynomial.

$$\zeta = \frac{-\ln OS}{\sqrt{\pi^2 + \ln^2 OS}}, \quad \sigma = \frac{4}{T_s}, \quad \omega_n = \frac{\sigma}{\zeta}$$

symbol		meaning	SI unit
`OS`	in	maximum overshoot (write 10% or 0.1)	,
`Ts`	in	2 % settling time in seconds	,
`zeta`	out	damping ratio	,
`wn`	out	natural frequency	rad/s
`sigma`	out	real part magnitude 4/Ts	,
`wd`	out	imaginary part of the poles	rad/s
`char_poly`	out	s^2 + 2 sigma s + wn^2	,

Applies when: 0 < OS < 100 %; Ts > 0.

```
$ control-systems solve ctrl_specs_to_poles OS=0.1630335348 Ts=2
zeta = 0.5
wn = 4 rad/s
sigma = 2
wd = 3.464101615 rad/s
char_poly = [1.0,4.0,15.9999999982]
  by: Desired closed-loop poles from overshoot and settling time  (zeta from the overshoot
formula; sigma = 4/Ts)
  OS = 0.1630335348
  Ts = 2
  assumes: 0 < OS < 100 %; Ts > 0
```

What the worked example means: This calculation determines the desired closed-loop pole locations based on specified transient response requirements. The damping ratio 0.5 is derived from the maximum overshoot 0.163034, while the real part of the poles, represented by sigma 2, is calculated as four divided by the two percent settling time 2. The natural frequency 4 rad/s in radians per second follows from these values, defining the characteristic polynomial s squared plus two sigma s plus wn squared.

Discretization (12)

Backward Euler (backward difference) discretization, `ctrl_backward_euler`

The discrete transfer function obtained by substituting s = (z - 1)/(T z) into G(s).

$$s \leftarrow \frac{z - 1}{T z}$$

symbol		meaning	SI unit
`num`	in	numerator in s	,
`den`	in	denominator in s	,
`T`	in	sampling period in seconds	,
`numz`	out	numerator in z (padded, normalised)	,
`denz`	out	denominator in z (leading coefficient 1)	,

Applies when: proper G(s); T > 0.

```
$ control-systems solve ctrl_backward_euler "num=[1]" "den=[1, 1]" T=0.1
numz = [0.0909090909091,0.0]
denz = [1.0,-0.909090909091]
  by: Backward Euler (backward difference) discretization  (polynomial substitution s = (z -
1)/(T z))
  T = 0.1
  assumes: proper G(s); T > 0
```

What the worked example means: This calculation applies the backward Euler discretization method by substituting s with (z - 1) divided by T z in the continuous-time transfer function G(s). The inputs are the numerator polynomial [1], the denominator polynomial [1, 1], and the sampling period 0.1. The resulting output provides the numerator [0.0909091, 0] and denominator [1, -0.909091] of the equivalent discrete-time transfer function, where the denominator is normalized to have a leading coefficient of one.

Continuous equivalent of a discrete pole (s = ln z / T), `ctrl_z_to_s_pole`

The s-plane pole s = ln(z)/T (principal branch) of a discrete pole z = z_re + j z_im, with its natural frequency and damping ratio.

$$s = \frac{\ln z}{T}$$

symbol		meaning	SI unit
`z_re`	in	real part of the discrete pole	,
`z_im`	in	imaginary part of the discrete pole (>= 0)	,
`T`	in	sampling period in seconds	,
`s_re`	out	real part of s (1/s)	,
`s_im`	out	imaginary part of s (rad/s)	,
`wn`	out	natural frequency	rad/s
`zeta`	out	damping ratio	,

Applies when: T > 0; z not on the negative real axis.

```
$ control-systems solve ctrl_z_to_s_pole z_re=0.5 z_im=0 T=0.1
s_re = -6.931471806
s_im = 0
wn = 6.931471806 rad/s
zeta = 1
  by: Continuous equivalent of a discrete pole (s = ln z / T)  (s = (ln|z| + j arg z)/T)
  z_re = 0.5
  z_im = 0
  T = 0.1
  assumes: T > 0; z not on the negative real axis
```

What the worked example means: This calculation converts the discrete pole defined by real part 0.5 and imaginary part 0 into its continuous time equivalent using sampling period 0.1. The resulting s-plane pole has a real component of -6.93147 and an imaginary component of 0. These values determine the system's dynamics, specifically yielding a natural frequency of 6.93147 rad/s radians per second and a damping ratio of 1, which describe how quickly oscillations decay and their inherent speed.

DC gain and final value of a discrete transfer function, `ctrl_discrete_dc_gain`

G(1) = sum(num)/sum(den) of a stable discrete transfer function G(z), and the final value of its response to a step of size A.

$$y(\infty) = A\,G(1)$$

symbol		meaning	SI unit
`num`	in	numerator coefficients in z, highest power first	,
`den`	in	denominator coefficients in z, highest power first	,
`A`	in	step size (default 1) (optional)	,
`dc_gain`	out	DC gain G(1)	,
`final_value`	out	final value of the step response	,

Applies when: proper G(z); every pole inside the unit circle.

```
$ control-systems solve ctrl_discrete_dc_gain "num=[0.5]" "den=[1, -0.5]"
dc_gain = 1
final_value = 1
  by: DC gain and final value of a discrete transfer function  (G(1) after a stability check)
  assumes: proper G(z); every pole inside the unit circle
```

In the worked example, num = [0.5], den = [1, -0.5] go through the equation above and give dc_gain = 1, final_value = 1. It applies only when proper G(z); every pole inside the unit circle: outside that it says what is wrong instead of returning a number.

Discrete PID in velocity (incremental) form, `ctrl_discrete_pid`

Coefficients of u[k] = u[k-1] + q0 e[k] + q1 e[k-1] + q2 e[k-2] for a PID with backward-difference integral and derivative: q0 = Kp + Ki T + Kd/T, q1 = -Kp - 2 Kd/T, q2 = Kd/T.

$$\Delta u_k = (K_p + K_i T + \tfrac{K_d}{T})e_k - (K_p + \tfrac{2K_d}{T})e_{k-1} + \tfrac{K_d}{T}e_{k-2}$$

symbol		meaning	SI unit
`Kp`	in	proportional gain	,
`Ki`	in	integral gain	,
`Kd`	in	derivative gain	,
`T`	in	sampling period in seconds	,
`q0`	out	coefficient of e[k]	,
`q1`	out	coefficient of e[k-1]	,
`q2`	out	coefficient of e[k-2]	,

Applies when: T > 0; Ki, Kd >= 0.

```
$ control-systems solve ctrl_discrete_pid Kp=2 Ki=1 Kd=0.5 T=0.1
q0 = 7.1
q1 = -12
q2 = 5
  by: Discrete PID in velocity (incremental) form  (difference of the positional backward-
difference PID)
  Kp = 2
  Ki = 1
  Kd = 0.5
  T = 0.1
  assumes: T > 0; Ki, Kd >= 0
```

What the worked example means: This calculation determines the three coefficients needed to implement a discrete PID controller in velocity form. Using the proportional, integral, and derivative gains along with the sampling period, it computes the weights applied to the current error and the two previous error samples. The resulting coefficient for the current error is 7.1, for the previous error is -12, and for the error before that is 5. These values allow the controller output to be updated incrementally based on recent error history.

First-order-hold (triangle-hold) discretization, `ctrl_c2d_foh`

The ramp-invariant (triangle-hold, non-causal first-order hold) discrete equivalent of a proper G(s) sampled every T seconds.

$$B_d = \Gamma_1 - \Gamma_2 + \Phi\Gamma_2, \quad D_d = D + C\Gamma_2$$

symbol		meaning	SI unit
`num`	in	numerator in s	,
`den`	in	denominator in s	,
`T`	in	sampling period in seconds	,
`numz`	out	numerator in z (padded, normalised)	,
`denz`	out	denominator in z (leading coefficient 1)	,

Applies when: proper G(s); T > 0.

```
$ control-systems solve ctrl_c2d_foh "num=[1]" "den=[1, 0]" T=1
numz = [0.5,0.5]
denz = [1.0,-1.0]
  by: First-order-hold (triangle-hold) discretization  (exponential of the augmented matrix [[A
T, B T, 0], [0, 0, 1], [0, 0, 0]])
  T = 1
  assumes: proper G(s); T > 0
```

What the worked example means: This calculation determines the discrete-time transfer function using the first-order hold method, which assumes a triangular input shape between samples. It takes the continuous numerator [1] and denominator [1, 0] along with the sampling period 1 to produce the normalized z-domain coefficients. The resulting output coefficients [0.5, 0.5] and [1, -1] describe how the digital system approximates the original analog behavior under piecewise linear inputs.

Forward Euler (forward difference) discretization, `ctrl_forward_euler`

The discrete transfer function obtained by substituting s = (z - 1)/T into G(s).

$$s \leftarrow \frac{z-1}{T}$$

symbol		meaning	SI unit
`num`	in	numerator in s	,
`den`	in	denominator in s	,
`T`	in	sampling period in seconds	,
`numz`	out	numerator in z (padded, normalised)	,
`denz`	out	denominator in z (leading coefficient 1)	,

Applies when: proper G(s); T > 0.

```
$ control-systems solve ctrl_forward_euler "num=[1]" "den=[1, 1]" T=0.1
numz = [0.0,0.1]
denz = [1.0,-0.9]
  by: Forward Euler (forward difference) discretization  (polynomial substitution s = (z -
1)/T)
  T = 0.1
  assumes: proper G(s); T > 0
```

What the worked example means: This algorithm converts a continuous-time transfer function into its discrete-time equivalent using the forward Euler method. It takes the continuous numerator [1] and denominator [1, 1] along with the sampling period 0.1. By replacing every instance of s with the expression involving z, it produces a new rational polynomial in the z-domain. The resulting numerator coefficients are listed as [0, 0.1] and the denominator coefficients as [1, -0.9]. This approximation assumes that the sampling period is small enough relative to the system dynamics for the linear approximation of the derivative to hold true.

Mapping an s-plane pole to the z-plane (z = e^{sT}), `ctrl_s_to_z_pole`

The discrete pole z = e^{sT} of a continuous pole s = s_re + j s_im sampled with period T: real and imaginary parts, magnitude e^{s_re T} and angle s_im T.

$$z = e^{sT} = e^{\sigma T} \angle \omega T$$

symbol		meaning	SI unit
`s_re`	in	real part of the s-plane pole (1/s)	,
`s_im`	in	imaginary part of the pole, >= 0 (rad/s)	,
`T`	in	sampling period in seconds	,
`z_re`	out	real part of z	,
`z_im`	out	imaginary part of z	,
`z_mag`	out	|z| = e^{s_re T}	,
`z_angle`	out	angle of z in radians, s_im T	,

Applies when: T > 0; 0 <= s_im T < pi.

```
$ control-systems solve ctrl_s_to_z_pole s_re=-2 s_im=0 T=0.1
z_re = 0.8187307531
z_im = 0
z_mag = 0.8187307531
z_angle = 0
  by: Mapping an s-plane pole to the z-plane (z = e^{sT})  (z = e^{s_re T}(cos s_im T + j sin
s_im T))
  s_re = -2
  s_im = 0
  T = 0.1
  assumes: T > 0; 0 <= s_im T < pi
```

What the worked example means: This calculation maps a continuous-time pole s to its discrete-time equivalent z using the sampling period T. The resulting complex number z is defined by its magnitude, which equals e raised to the product of the real part of s and T, and its angle, which equals the product of the imaginary part of s and T. For the given inputs, the computed magnitude is 0.818731 and the angle is 0. These values determine the location of the pole in the z-plane, where the real component is 0.818731 and the imaginary component is 0.

Step response of a discrete transfer function at sample k, `ctrl_discrete_step`

y[k] of G(z) = numz/denz after a step of size A applied at k = 0, by the difference equation.

$$y[k] = \sum_{i} b_i u[k-i] - \sum_{i \geq 1} a_i y[k-i]$$

symbol		meaning	SI unit
`numz`	in	numerator in z, highest power first	,
`denz`	in	denominator in z, highest power first	,
`k`	in	sample index (0 to 500)	,
`A`	in	step size (default 1) (optional)	,
`y`	out	output at sample k	,

Applies when: proper (causal) G(z); 0 <= k <= 500.

```
$ control-systems solve ctrl_discrete_step "numz=[0.5]" "denz=[1, -0.5]" k=2
y = 0.75
  by: Step response of a discrete transfer function at sample k  (recursion y[j] = sum b_i u[j-i] - sum a_i y[j-i])
  assumes: proper (causal) G(z); 0 <= k <= 500
```

In the worked example, numz = [0.5], denz = [1, -0.5], k = 2 go through the equation above and give y = 0.75. It applies only when proper (causal) G(z); 0 <= k <= 500: outside that it says what is wrong instead of returning a number.

Tustin (bilinear) discretization, `ctrl_tustin`

The discrete transfer function obtained by substituting s = (2/T)(z - 1)/(z + 1) into G(s).

$$s \leftarrow \frac{2}{T}\,\frac{z-1}{z+1}$$

symbol		meaning	SI unit
`num`	in	numerator in s	,
`den`	in	denominator in s	,
`T`	in	sampling period in seconds	,
`numz`	out	numerator in z (padded, normalised)	,
`denz`	out	denominator in z (leading coefficient 1)	,

Applies when: proper G(s); T > 0.

```
$ control-systems solve ctrl_tustin "num=[1]" "den=[1, 1]" T=0.1
numz = [0.047619047619,0.047619047619]
denz = [1.0,-0.904761904762]
  by: Tustin (bilinear) discretization  (polynomial substitution, multiplied through by (z + 1)^n)
  T = 0.1
  assumes: proper G(s); T > 0
```

What the worked example means: This calculation applies the Tustin bilinear transformation to convert a continuous transfer function into a discrete one. By substituting the sampling period 0.1 seconds and the original coefficients into the formula, this gives the new polynomial expressions in z. The resulting numerator [0.047619, 0.047619] and normalized denominator [1, -0.904762] represent the system's behavior at the specified sampling rate.

Tustin discretization with frequency prewarping, `ctrl_tustin_prewarp`

The bilinear transform s = (w/tan(w T/2)) (z - 1)/(z + 1), which matches the continuous frequency response exactly at the prewarping frequency w.

$$s \leftarrow \frac{\omega}{\tan(\omega T/2)}\,\frac{z-1}{z+1}$$

symbol		meaning	SI unit
`num`	in	numerator in s	,

symbol		meaning	SI unit
`den`	in	denominator in s	,
`T`	in	sampling period in seconds	,
`w`	in	prewarping frequency in rad/s	,
`numz`	out	numerator in z (padded, normalised)	,
`denz`	out	denominator in z (leading coefficient 1)	,

Applies when: proper G(s); 0 < w T < pi.

```
$ control-systems solve ctrl_tustin_prewarp "num=[1]" "den=[1, 1]" T=1 w=1
numz = [0.353296003487,0.353296003487]
denz = [1.0,-0.293407993026]
  by: Tustin discretization with frequency prewarping  (polynomial substitution with the
prewarped constant)
  T = 1
  w = 1
  assumes: proper G(s); 0 < w T < pi
```

What the worked example means: This calculation converts the continuous system defined by numerator [1] and denominator [1, 1] into a discrete equivalent using sampling period 1 and prewarping frequency 1. The resulting discrete numerator coefficients are [0.353296, 0.353296] and the normalized denominator coefficients are [1, -0.293408], ensuring the frequency response matches exactly at 1.

Zero-order-hold discretization of a first-order system, `ctrl_zoh_first_order`

G(z) = b/(z - a) with a = e^{-T/tau} and b = K (1 - a): the exact step-invariant (ZOH) equivalent of K/(tau s + 1) sampled every T seconds.

$$G(z) = \frac{K(1 - e^{-T/\tau})}{z - e^{-T/\tau}}$$

symbol		meaning	SI unit
`K`	in	DC gain	,
`tau`	in	time constant in seconds	,
`T`	in	sampling period in seconds	,
`b`	out	numerator coefficient b	,
`a`	out	discrete pole a	,

Applies when: tau > 0; T > 0.

```
$ control-systems solve ctrl_zoh_first_order K=2 tau=1 T=0.5
b = 0.7869386806
a = 0.6065306597
  by: Zero-order-hold discretization of a first-order system  (a = exp(-T/tau), b = K (1 - a))
  K = 2
  tau = 1
  T = 0.5
  assumes: tau > 0; T > 0
```

What the worked example means: This calculation determines the parameters for the zero-order-hold discretization of a first-order system with DC gain 2 and time constant 1. By sampling every 0.5 seconds, the continuous transfer function is mapped to a discrete form where the pole a equals 0.606531 and the numerator coefficient b equals 0.786939. These values define the dynamic response of the digital controller or simulation model derived from the physical plant.

Zero-order-hold discretization of a transfer function, `ctrl_c2d_zoh`

The step-invariant (ZOH) discrete equivalent G(z) = (1 - z^{-1}) Z{G(s)/s} of a proper G(s) sampled every T seconds.

$$A_d = e^{AT},\ B_d = \int_0^T e^{A\tau} d\tau\, B$$

symbol		meaning	SI unit
`num`	in	numerator in s	,
`den`	in	denominator in s	,
`T`	in	sampling period in seconds	,
`numz`	out	numerator in z (padded, normalised)	,
`denz`	out	denominator in z (leading coefficient 1)	,

Applies when: proper G(s); T > 0.

```
$ control-systems solve ctrl_c2d_zoh "num=[1]" "den=[1, 1]" T=0.5
numz = [0.0,0.393469340287]
denz = [1.0,-0.606530659713]
  by: Zero-order-hold discretization of a transfer function  (exponential of [[A, B], [0, 0]] T
in state space, then Faddeev-LeVerrier)
  T = 0.5
  assumes: proper G(s); T > 0
```

What the worked example means: This process converts a continuous-time transfer function into its step-invariant discrete counterpart. It uses the numerator in s [1] and denominator in s [1, 1] with a sampling period of 0.5 seconds. The resulting discrete numerator is [0, 0.393469] and the normalized discrete denominator is [1, -0.606531], representing the system behavior when held constant between samples.

Estimation (4)

One step of a scalar Kalman filter, `ctrl_kalman_scalar_step`

Predict (x- = a x, P- = a^2 P + q) and update with a measurement z of variance r (gain P-/(P- + r)) for x_k = a x_{k-1} + w, z_k = x_k + v.

$$K = \frac{P^-}{P^- + r},\quad \hat{x} = \hat{x}^- + K(z - \hat{x}^-),\quad P = (1 - K)P^-$$

symbol		meaning	SI unit
`x`	in	previous estimate	,
`P`	in	previous error variance	,

symbol		meaning	SI unit
`a`	in	state transition coefficient	,
`q`	in	process noise variance	,
`r`	in	measurement noise variance	,
`z`	in	new measurement	,
`x_pred`	out	predicted estimate	,
`P_pred`	out	predicted variance	,
`gain`	out	Kalman gain	,
`x_post`	out	updated estimate	,
`P_post`	out	updated variance	,

Applies when: P, q >= 0; r > 0.

```
$ control-systems solve ctrl_kalman_scalar_step x=0 P=1 a=1 q=1 r=2 z=4
x_pred = 0
P_pred = 2
gain = 0.5
x_post = 2
P_post = 1
  by: One step of a scalar Kalman filter  (Kalman predict and update equations)
  x = 0
  P = 1
  a = 1
  q = 1
  r = 2
  z = 4
  assumes: P, q >= 0; r > 0
```

What the worked example means: This calculation performs one step of a scalar Kalman filter, combining prediction and update phases. Starting from the previous estimate 0 and error variance 1, the system predicts the next state using transition coefficient 1 and process noise 1, yielding predicted values 0 and 2. The filter then incorporates the new measurement 4, which has noise variance 2, to compute the optimal gain 0.5. This gain balances trust in the prediction versus the measurement, resulting in the final updated estimate 2 and reduced uncertainty 1.

Steady-state discrete Kalman filter gain, `ctrl_discrete_kalman_gain`

For x_{k+1} = A x_k + w (covariance Q), y_k = C x_k + v (variance R): the steady prior covariance P from the discrete filter Riccati equation and the measurement-update gain K = P C^T (C P C^T + R)^{-1}.

$$P = APA^T - APC^T(CPC^T + R)^{-1}CPA^T + Q, \quad K = PC^T(CPC^T + R)^{-1}$$

symbol		meaning	SI unit
`A`	in	discrete state matrix (n x n)	,
`C`	in	output vector	,
`Q`	in	process noise covariance (symmetric positive definite)	,
`R`	in	measurement noise variance (> 0)	,
`K`	out	steady Kalman gain	,

symbol		meaning	SI unit
`P_prior`	out	steady prior error covariance	,

Applies when: (A, C) observable; Q positive definite; R > 0.

```
$ control-systems solve ctrl_discrete_kalman_gain A=1 C=1 Q=1 R=1
K = [0.61803398875]
P_prior = [[1.61803398875]]
  by: Steady-state discrete Kalman filter gain  (the dual discrete Riccati equation by Hewer's
iteration)
  R = 1
  assumes: (A, C) observable; Q positive definite; R > 0
```

What the worked example means: This calculation determines the optimal measurement update weights for a discrete linear system. Using Hewer's iteration to solve the dual Riccati equation, it finds the steady prior error covariance [[1.61803]] and the corresponding gain [0.618034]. The gain dictates how much weight the filter places on new measurements versus its internal prediction, balancing process noise [1] against measurement noise 1.

Steady-state gain of a scalar Kalman filter, `ctrl_kalman_scalar_steady`

For x_k = a x_{k-1} + w (variance q) and z_k = c x_k + v (variance r): the steady prior variance P (discrete Riccati equation), the steady Kalman gain and the posterior variance.

$$P = a^2 P - \frac{a^2 c^2 P^2}{c^2 P + r} + q, \quad K = \frac{P c}{c^2 P + r}$$

symbol		meaning	SI unit
`a`	in	state transition coefficient	,
`c`	in	measurement coefficient	,
`q`	in	process noise variance	,
`r`	in	measurement noise variance	,
`P_prior`	out	steady prior error variance	,
`gain`	out	steady Kalman gain	,
`P_post`	out	steady posterior error variance	,

Applies when: c != 0; q, r > 0.

```
$ control-systems solve ctrl_kalman_scalar_steady a=1 c=1 q=1 r=1
P_prior = 1.618033989
gain = 0.6180339888
P_post = 0.6180339888
  by: Steady-state gain of a scalar Kalman filter  (positive root of c^2 P^2 + (r - a^2 r - q
c^2) P - q r = 0)
  a = 1
  c = 1
  q = 1
  r = 1
  assumes: c != 0; q, r > 0
```

What the worked example means: This calculation determines the long-term behavior of a scalar Kalman filter by solving the discrete Riccati equation for the steady-state prior error variance, which results in 1.61803. Using

this variance, the algorithm computes the optimal Kalman gain, found here to be 0.618034, which balances the trust placed in the measurement versus the prediction. The resulting posterior error variance, representing the uncertainty after updating with a new measurement, is 0.618034. These values indicate how effectively the filter reduces noise given the specific process and measurement characteristics.

Steady-state Kalman-Bucy filter gain, `ctrl_kalman_bucy`

For x' = A x + w (covariance W), y = C x + v (variance V): the steady error covariance P from the filter Riccati equation A P + P A^T - P C^T V^{-1} C P + W = 0, the gain L = P C^T / V and the estimator poles (eigenvalues of A - L C).

$$AP + PA^T - PC^TV^{-1}CP + W = 0, \quad L = PC^TV^{-1}$$

symbol		meaning	SI unit
`A`	in	state matrix (n x n)	,
`C`	in	output vector	,
`W`	in	process noise covariance (symmetric positive definite)	,
`V`	in	measurement noise variance (> 0)	,
`L`	out	Kalman gain (column)	,
`P`	out	steady error covariance	,
`poles`	out	estimator poles	,

Applies when: (A, C) observable; W positive definite; V > 0.

```
$ control-systems solve ctrl_kalman_bucy "A=[[0, 1], [0, 0]]" "C=[1, 0]" "W=[[1, 0], [0, 1]]"
V=1
L = [1.73205080757,1.0]
P = [[1.73205080757,1.0],[1.0,1.73205080757]]
poles = [{"re":-0.866025403784,"im":-0.5},{"re":-0.866025403784,"im":0.5}]
  by: Steady-state Kalman-Bucy filter gain  (the dual LQR problem (A^T, C^T, W, V) by Newton-
Kleinman iteration)
  V = 1
  assumes: (A, C) observable; W positive definite; V > 0
```

What the worked example means: The steady-state error covariance [[1.73205, 1], [1, 1.73205]] is computed by solving the filter Riccati equation using Newton-Kleinman iteration on the dual LQR problem. This covariance determines the optimal gain [1.73205, 1], which balances process noise uncertainty [1 0; 0 1] against measurement noise variance 1. The resulting closed-loop estimator dynamics are governed by the matrix A minus the product of the gain and output matrix, producing the estimated poles [-0.866025 - 0.5i, -0.866025 + 0.5i] that dictate the convergence rate of the state estimate.

Frequency response (8)

-3 dB bandwidth of a transfer function, `ctrl_bandwidth`

The lowest frequency at which |G(jw)| falls to |G(0)|/sqrt(2) for a stable, strictly proper G(s), and the DC gain.

$$|G(j\omega_B)| = \frac{|G(0)|}{\sqrt{2}}$$

symbol		meaning	SI unit
`num`	in	numerator	,
`den`	in	denominator	,
`wb`	out	-3 dB bandwidth	rad/s
`dc_gain`	out	DC gain	,

Applies when: stable; strictly proper; nonzero DC gain.

```
$ control-systems solve ctrl_bandwidth "num=[1]" "den=[1, 1]"
wb = 1 rad/s
dc_gain = 1
  by: -3 dB bandwidth of a transfer function  (smallest positive root of 2|N(jw)|^2 - G(0)^2
|D(jw)|^2)
  assumes: stable; strictly proper; nonzero DC gain
```

What the worked example means: This calculation determines the frequency range where the system responds effectively. The result, 1 rad/s rad/s, represents the point where the magnitude drops to about seventy percent of its low-frequency value, which is 1. This metric indicates how quickly the system can react to changes in input signals.

Bandwidth of a standard second-order system, `ctrl_bandwidth_second`

The -3 dB bandwidth wn sqrt(1 - 2 zeta^2 + sqrt(4 zeta^4 - 4 zeta^2 + 2)) of wn^2/(s^2 + 2 zeta wn s + wn^2).

$$\omega_B = \omega_n \sqrt{1 - 2\zeta^2 + \sqrt{4\zeta^4 - 4\zeta^2 + 2}}$$

symbol		meaning	SI unit
`zeta`	in	damping ratio (> 0)	,
`wn`	in	natural frequency in rad/s	,
`wb`	out	-3 dB bandwidth	rad/s

Applies when: zeta > 0; wn > 0.

```
$ control-systems solve ctrl_bandwidth_second zeta=0.7071067812 wn=1
wb = 1 rad/s
  by: Bandwidth of a standard second-order system  (solve |G(jw)| = 1/sqrt(2))
  zeta = 0.7071067812
  wn = 1
  assumes: zeta > 0; wn > 0
```

What the worked example means: This calculation determines the -3 dB bandwidth for a standard second-order system. It uses the provided natural frequency and damping ratio to find the frequency at which the magnitude of the transfer function drops to one over the square root of two times its low-frequency value. The resulting bandwidth is 1 rad/s rad/s, indicating how quickly the system responds to input changes before attenuation becomes significant.

Bode magnitude and phase at one frequency, `ctrl_bode_point`

|G(jw)|, 20 log10 |G(jw)| and the phase of G(jw) (principal value) of a transfer function at the angular frequency w.

$$|G(j\omega)|_{dB} = 20\log_{10}|G(j\omega)|$$

symbol		meaning	SI unit
`num`	in	numerator coefficients	,
`den`	in	denominator coefficients	,
`w`	in	angular frequency in rad/s	,
`magnitude`	out	|G(jw)|	,
`magnitude_db`	out	magnitude in dB	,
`phase_deg`	out	phase in degrees, (-180, 180]	,

Applies when: proper G(s); w > 0; not a pole on the imaginary axis.

```
$ control-systems solve ctrl_bode_point "num=[10]" "den=[1, 1, 0]" w=1
magnitude = 7.071067812
magnitude_db = 16.98970004
phase_deg = -135
  by: Bode magnitude and phase at one frequency  (G(jw) by complex evaluation)
  w = 1
  assumes: proper G(s); w > 0; not a pole on the imaginary axis
```

What the worked example means: This calculation evaluates the transfer function at a specific angular frequency to determine its gain and phase shift. The absolute magnitude of the complex response is 7.07107, which converts to 16.9897 decibels using the logarithmic scale. The phase angle, representing the time delay or lead in the system's output relative to its input, is found to be -135 degrees. These values describe how the system modifies a sinusoidal signal at that particular frequency.

Frequency response of a first-order lag, `ctrl_first_order_freq`

Magnitude K/sqrt(1 + (w tau)^2), its value in dB and the phase -atan(w tau) of K/(tau s + 1) at the angular frequency w.

$$|G(j\omega)| = \frac{K}{\sqrt{1+\omega^2\tau^2}}, \quad \angle G = -\arctan(\omega\tau)$$

symbol		meaning	SI unit
`K`	in	DC gain (positive)	,
`tau`	in	time constant in seconds	,
`w`	in	angular frequency in rad/s	,
`magnitude`	out	gain |G(jw)|	,
`magnitude_db`	out	gain in dB	,
`phase_deg`	out	phase in degrees	,

Applies when: K > 0; tau > 0; w >= 0.

```
$ control-systems solve ctrl_first_order_freq K=10 tau=1 w=1
magnitude = 7.071067812
magnitude_db = 16.98970004
phase_deg = -45
  by: Frequency response of a first-order lag  (G(jw) = K/(1 + j w tau))
  K = 10
  tau = 1
  w = 1
  assumes: K > 0; tau > 0; w >= 0
```

What the worked example means: This calculation determines the frequency response of a first-order lag system defined by its DC gain 10 and time constant 1. By evaluating the transfer function at the angular frequency 1, this gives the linear magnitude 7.07107, which represents how much the signal amplitude is scaled. This value is also expressed in decibels as 16.9897 for logarithmic comparison. Additionally, the output includes the phase shift -45, indicating the delay introduced by the system at this specific frequency relative to the input.

Frequency response of a standard second-order system, `ctrl_second_order_freq`

Magnitude, dB gain and phase (between 0 and -180 degrees) of wn^2/(s^2 + 2 zeta wn s + wn^2) at the angular frequency w.

$$|G| = \frac{\omega_n^2}{\sqrt{(\omega_n^2 - \omega^2)^2 + (2\zeta\omega_n\omega)^2}}$$

symbol		meaning	SI unit
`zeta`	in	damping ratio	,
`wn`	in	natural frequency in rad/s	,
`w`	in	angular frequency in rad/s	,
`magnitude`	out	|G(jw)|	,
`magnitude_db`	out	magnitude in dB	,
`phase_deg`	out	phase in degrees, [-180, 0]	,

Applies when: zeta >= 0; wn > 0; w > 0.

```
$ control-systems solve ctrl_second_order_freq zeta=0.5 wn=10 w=10
magnitude = 1
magnitude_db = 0
phase_deg = -90
  by: Frequency response of a standard second-order system  (G(jw) = wn^2/(wn^2 - w^2 + 2 j
zeta wn w))
  zeta = 0.5
  wn = 10
  w = 10
  assumes: zeta >= 0; wn > 0; w > 0
```

What the worked example means: This calculation determines the frequency response of a standard second-order system at the angular frequency 10. Using the natural frequency 10 and damping ratio 0.5, it computes the gain magnitude as 1, its decibel equivalent as 0, and the phase shift as -90 degrees. These values describe how the system amplifies or attenuates signals and shifts their timing at this specific frequency.

H-infinity norm (peak gain) of a stable transfer function, `ctrl_hinf_norm`

The peak gain sup_w |G(jw)| of a stable proper G(s), in ratio and dB, and the frequency where it occurs (0 when the peak is the DC gain).

$$\|G\|_\infty = \sup_\omega |G(j\omega)|$$

symbol		meaning	SI unit
`num`	in	numerator	,
`den`	in	denominator	,
`hinf`	out	H-infinity norm (peak gain)	,
`hinf_db`	out	peak gain in dB	,
`w_peak`	out	frequency of the peak	rad/s

Applies when: stable; proper; peak at a finite frequency.

```
$ control-systems solve ctrl_hinf_norm "num=[1]" "den=[1, 1]"
hinf = 1
hinf_db = 0
w_peak = 0 rad/s
  by: H-infinity norm (peak gain) of a stable transfer function  (stationary points of
|N(jw)|^2/|D(jw)|^2 (positive real roots of f' g - f g') and w = 0)
  assumes: stable; proper; peak at a finite frequency
```

What the worked example means: This calculation determines the maximum magnitude response of a stable transfer function across all frequencies. By analyzing the stationary points of the squared magnitude ratio and including direct current gain, the method identifies the specific frequency where the system exhibits its highest amplification. The resulting peak gain value of 1 indicates the worst-case signal amplification factor, while the associated frequency of 0 rad/s radians per second specifies where this resonance or peak occurs in the spectrum.

H2 norm of a stable transfer function, `ctrl_h2_norm`

||G||_2 = sqrt((1/(2 pi)) integral of |G(jw)|^2 dw) of a stable, strictly proper G(s), computed exactly as B^T Wo B from the observability Gramian.

$$\|G\|_2^2 = \frac{1}{2\pi}\int_{-\infty}^{\infty} |G(j\omega)|^2 d\omega = B^T W_o B$$

symbol		meaning	SI unit
`num`	in	numerator	,
`den`	in	denominator	,
`h2`	out	H2 norm	,
`h2_squared`	out	squared H2 norm	,

Applies when: stable; strictly proper.

```
$ control-systems solve ctrl_h2_norm "num=[1]" "den=[1, 1]"
h2 = 0.7071067812
h2_squared = 0.5
  by: H2 norm of a stable transfer function  (Lyapunov equation of the controllable canonical
form in exact fractions)
  assumes: stable; strictly proper
```

What the worked example means: This calculation determines the H2 norm for the transfer function defined by numerator [1] and denominator [1, 1]. The method solves the Lyapunov equation for the observability Gramian in exact fractions to compute the squared norm as B transpose W sub o times B. The final result, 0.707107, represents the root mean square response of the system to white noise input.

Resonant peak and resonant frequency, `ctrl_resonant_peak`

Peak magnitude Mr = 1/(2 zeta sqrt(1 - zeta^2)) and resonant frequency wr = wn sqrt(1 - 2 zeta^2) of a standard second-order system.

$$M_r = \frac{1}{2\zeta\sqrt{1-\zeta^2}}, \quad \omega_r = \omega_n\sqrt{1-2\zeta^2}$$

symbol		meaning	SI unit
`zeta`	in	damping ratio (0 < zeta < 0.7071)	,
`wn`	in	natural frequency in rad/s	,
`Mr`	out	resonant peak (ratio)	,
`Mr_db`	out	resonant peak in dB	,
`wr`	out	resonant frequency	rad/s

Applies when: 0 < zeta < 1/sqrt(2).

```
$ control-systems solve ctrl_resonant_peak zeta=0.5 wn=10
Mr = 1.154700538
Mr_db = 1.249387366
wr = 7.071067812 rad/s
  by: Resonant peak and resonant frequency  (maximum of |G(jw)|)
  zeta = 0.5
  wn = 10
  assumes: 0 < zeta < 1/sqrt(2)
```

What the worked example means: This calculation determines the maximum magnitude of the frequency response for a standard second-order system. Using the damping ratio 0.5 and natural frequency 10, the resonant peak Mr is found to be 1.1547, which corresponds to 1.24939 in decibels. The frequency at which this peak occurs, known as the resonant frequency wr, is calculated as 7.07107 rad/s. These values indicate how much the system amplifies signals near its natural frequency and exactly where that amplification happens.

Identification (3)

Damping ratio from the logarithmic decrement, `ctrl_log_decrement`

delta = ln(x1/xn)/n from two peak amplitudes n cycles apart, and zeta = delta/sqrt(4 pi^2 + delta^2).

$$\delta = \frac{1}{n}\ln\frac{x_1}{x_{n+1}}, \quad \zeta = \frac{\delta}{\sqrt{4\pi^2 + \delta^2}}$$

symbol		meaning	SI unit
`x1`	in	first peak amplitude (measured from the final value)	,
`xn`	in	peak amplitude n cycles later	,
`n`	in	number of cycles between the peaks	,
`delta`	out	logarithmic decrement per cycle	,
`zeta`	out	damping ratio	,

Applies when: 0 < xn < x1; n >= 1.

```
$ control-systems solve ctrl_log_decrement x1=1 xn=0.5 n=1
delta = 0.6931471806
zeta = 0.109652581
  by: Damping ratio from the logarithmic decrement  (ratio of successive peaks)
  x1 = 1
  xn = 0.5
  assumes: 0 < xn < x1; n >= 1
```

What the worked example means: This calculation determines the system's damping characteristics by measuring how quickly oscillations decay. The logarithmic decrement, calculated as 0.693147, represents the natural logarithm of the ratio between two successive peak amplitudes, specifically 1 and 0.5, normalized by the number of cycles 1 separating them. Using this value, the damping ratio 0.109653 is computed to quantify the energy dissipation relative to critical damping. A higher damping ratio indicates a more heavily damped system where oscillations die out faster, while a lower ratio suggests underdamped behavior with persistent ringing.

First-order-plus-dead-time fit by Smith's two-point method, `ctrl_fopdt_smith`

K = dy/du, tau = 1.5 (t63 - t28) and theta = t63 - tau from the times at which a step response reaches 28.3 % and 63.2 % of its change.

$$\tau = 1.5(t_{63} - t_{28}), \quad \theta = t_{63} - \tau$$

symbol		meaning	SI unit
`t28`	in	time to 28.3 % of the change (s)	,
`t63`	in	time to 63.2 % of the change (s)	,
`dy`	in	total output change	,
`du`	in	input step size	,
`K`	out	process gain	,
`tau`	out	time constant	s
`theta`	out	dead time	s

Applies when: 0 < t28 < t63; theta >= 0.

```
$ control-systems solve ctrl_fopdt_smith t28=4 t63=10 dy=6 du=2
K = 3
tau = 9 s
theta = 1 s
  by: First-order-plus-dead-time fit by Smith's two-point method  (Smith's method)
  t28 = 4
  t63 = 10
  dy = 6
  du = 2
  assumes: 0 < t28 < t63; theta >= 0
```

What the worked example means: Smith's two-point method estimates a first-order plus dead-time model from step response data. Using the times to reach twenty-eight point three percent (4) and sixty-three point two percent (10) of the total change, the time constant is calculated as one and five times the difference between these times. The dead time is then found by subtracting the time constant from the later time. The process gain is simply the ratio of the output change to the input step size.

Second-order model from a step response, `ctrl_identify_second_order`

Gain K = y_final/A, percent overshoot, zeta and wn of a standard second-order model from the peak value, the final value and the peak time of a measured step response.

$$\omega_n = \frac{\pi}{T_p\sqrt{1-\zeta^2}}$$

symbol		meaning	SI unit
`y_peak`	in	peak value of the response	,
`y_final`	in	final (steady) value	,
`t_peak`	in	time of the peak in seconds	,
`A`	in	step size (default 1) (optional)	,
`K`	out	DC gain	,
`PO`	out	percent overshoot (%)	,
`zeta`	out	damping ratio	,
`wn`	out	natural frequency	rad/s

Applies when: 0 < overshoot < 100 %; t_peak > 0.

```
$ control-systems solve ctrl_identify_second_order y_peak=1.163033535 y_final=1
t_peak=0.9068996821
K = 1
PO = 16.3033535
zeta = 0.4999999998
wn = 3.999999999 rad/s
  by: Second-order model from a step response  (zeta from the overshoot, wn = pi/(tp sqrt(1 -
zeta^2)))
  y_peak = 1.163033535
  y_final = 1
  t_peak = 0.9068996821
  assumes: 0 < overshoot < 100 %; t_peak > 0
```

What the worked example means: This calculation derives the parameters of a standard second-order model from measured step response characteristics. The DC gain is found by dividing the final steady-state value by the input step size, yielding 1. Percent overshoot is computed from the peak and final values as 16.3034 percent. Using this overshoot, the damping ratio is determined to be 0.5. Finally, the natural frequency is calculated using the peak time and damping ratio, resulting in 4 rad/s radians per second.

Modelling (1)

Speed transfer function of a DC motor, `ctrl_dc_motor`

omega(s)/V(s) = Kt/((J s + b)(L s + R) + Kt^2) of an armature-controlled DC motor (Ke = Kt in SI units): coefficients, DC gain and poles.

$$\frac{\Omega(s)}{V(s)} = \frac{K_t}{(Js+b)(Ls+R)+K_t^2}$$

symbol		meaning	SI unit
`J`	in	rotor inertia in kg*m^2	,
`b`	in	viscous friction in N*m*s	,
`Kt`	in	torque (= back-emf) constant in N*m/A	,
`R`	in	armature resistance in ohm	,
`L`	in	armature inductance in H	,
`num`	out	numerator	,
`den`	out	denominator [J L, b L + J R, b R + Kt^2]	,
`dc_gain`	out	speed per volt at steady state (rad/s per V)	,
`poles`	out	poles	,

Applies when: J, Kt, R, L > 0; b >= 0.

```
$ control-systems solve ctrl_dc_motor J=0.01 b=0.1 Kt=0.01 R=1 L=0.5
num = [0.01]
den = [0.005,0.06,0.1001]
dc_gain = 0.0999000999
poles = [-9.99749921826,-2.00250078174]
  by: Speed transfer function of a DC motor  (electrical and mechanical equations combined)
  J = 0.01
  b = 0.1
  Kt = 0.01
  R = 1
  L = 0.5
  assumes: J, Kt, R, L > 0; b >= 0
```

What the worked example means: This calculation determines the relationship between applied voltage and angular speed for an armature-controlled direct current motor. By combining electrical and mechanical equations, this gives the transfer function where the numerator is [0.01] and the denominator consists of coefficients representing inertial and resistive effects. The resulting steady-state speed per volt, or DC gain, is 0.0999001, while the system poles are located at [-9.9975, -2.0025]. These values indicate how quickly the motor responds to input changes and its ultimate rotational speed under constant voltage.

Optimal control (3)

Continuous-time LQR gain (Riccati equation), `ctrl_lqr`

The optimal state feedback u = -K x minimising the integral of x^T Q x + R u^2 for x' = A x + B u: the Riccati solution P, K = R^{-1} B^T P and the closed-loop poles.

$$A^T P + P A - P B R^{-1} B^T P + Q = 0, \quad K = R^{-1} B^T P$$

symbol		meaning	SI unit
`A`	in	state matrix (n x n)	,
`B`	in	input vector	,
`Q`	in	state weight (symmetric positive definite)	,
`R`	in	input weight (> 0)	,
`K`	out	optimal gain	,
`P`	out	Riccati solution	,
`poles`	out	closed-loop poles	,

Applies when: (A, B) controllable; Q symmetric positive definite; R > 0.

```
$ control-systems solve ctrl_lqr "A=[[0, 1], [0, 0]]" "B=[0, 1]" "Q=[[1, 0], [0, 1]]" R=1
K = [1.0,1.73205080757]
P = [[1.73205080757,1.0],[1.0,1.73205080757]]
poles = [{"re":-0.866025403784,"im":-0.5},{"re":-0.866025403784,"im":0.5}]
  by: Continuous-time LQR gain (Riccati equation)  (Newton-Kleinman iteration (a Lyapunov
equation per step) from a stabilising Ackermann gain)
  R = 1
  assumes: (A, B) controllable; Q symmetric positive definite; R > 0
```

What the worked example means: This calculation determines the continuous-time linear quadratic regulator gain [1, 1.73205] and the associated Riccati solution matrix [[1.73205, 1], [1, 1.73205]]. The process solves the algebraic Riccati equation using Newton-Kleinman iteration, starting from a stabilizing Ackermann gain. The resulting gain ensures that the control law u equals negative K times x minimizes the integral of the state weighting [1 0; 0 1] and input weighting 1 over time for the dynamics specified by [0 1; 0 0] and [0, 1]. The closed-loop poles [-0.866025 - 0.5i, -0.866025 + 0.5i] reflect the stability characteristics achieved by this optimal configuration.

Discrete-time LQR gain (discrete Riccati equation), `ctrl_dlqr`

The optimal state feedback u_k = -K x_k minimising the sum of x^T Q x + R u^2 for x_{k+1} = A x_k + B u_k: P from the discrete Riccati equation, K = (R + B^T P B)^{-1} B^T P A and the closed-loop poles.

$$P = A^T P A - A^T P B (R + B^T P B)^{-1} B^T P A + Q$$

symbol		meaning	SI unit
`A`	in	discrete state matrix (n x n)	,
`B`	in	input vector	,
`Q`	in	state weight (symmetric positive definite)	,
`R`	in	input weight (> 0)	,

symbol		meaning	SI unit
`K`	out	optimal gain	,
`P`	out	Riccati solution	,
`poles`	out	closed-loop poles	,

Applies when: (A, B) controllable; Q symmetric positive definite; R > 0.

```
$ control-systems solve ctrl_dlqr "A=[[1, 1], [0, 1]]" "B=[0, 1]" "Q=[[1, 0], [0, 1]]" R=1
K = [0.422082440385,1.2439288539]
P = [[2.94712296671,2.36920540709],[2.36920540709,4.613134261]]
poles = [{"re":0.378035573048,"im":-0.187730370457},{"re":0.378035573048,"im":0.187730370457}]
  by: Discrete-time LQR gain (discrete Riccati equation)  (Hewer's iteration (a discrete
Lyapunov equation per step) from the deadbeat gain)
  R = 1
  assumes: (A, B) controllable; Q symmetric positive definite; R > 0
```

In the worked example, A = [1 1; 0 1], B = [0, 1], Q = [1 0; 0 1], R = 1 go through the equation above and give K = [0.4220824404, 1.243928854], P = [[2.947122967, 2.369205407], [2.369205407, 4.613134261]], poles = [0.378035573-0.1877303705i, 0.378035573+0.1877303705i]. It applies only when (A, B) controllable; Q symmetric positive definite; R > 0: outside that it says what is wrong instead of returning a number.

LQR for a first-order system, `ctrl_scalar_lqr`

For x' = a x + b u minimising the integral of q x^2 + r u^2: the Riccati solution P, the optimal gain K = b P/r and the closed-loop pole -sqrt(a^2 + b^2 q/r).

$$2aP - \frac{b^2 P^2}{r} + q = 0, \quad K = \frac{bP}{r}$$

symbol		meaning	SI unit
`a`	in	system coefficient	,
`b`	in	input coefficient	,
`q`	in	state weight (> 0)	,
`r`	in	input weight (> 0)	,
`P`	out	Riccati solution	,
`K`	out	optimal feedback gain	,
`pole`	out	closed-loop pole a - b K	,

Applies when: b != 0; q, r > 0.

```
$ control-systems solve ctrl_scalar_lqr a=1 b=1 q=3 r=1
P = 3
K = 3
pole = -2
  by: LQR for a first-order system  (positive root of 2 a P - b^2 P^2 / r + q = 0)
  a = 1
  b = 1
  q = 3
  r = 1
  assumes: b != 0; q, r > 0
```

What the worked example means: This calculation determines the optimal feedback gain for a first-order linear system by solving the algebraic Riccati equation. Given the system coefficient 1, input coefficient 1, state weight 3, and control weight 1, the algorithm computes the positive root of the quadratic equation to find the Riccati solution 3. The optimal gain 3 is then derived from this solution, ensuring that the closed-loop system pole -2 places the system in the stable left half-plane while minimizing the weighted cost of state deviation and control effort.

PID tuning (9)

Closed-loop poles of a PID loop, `ctrl_pid_closed_loop`

The characteristic polynomial s D(s) + (Kd s^2 + Kp s + Ki) N(s), the closed-loop poles and stability of a plant N/D under a PID controller with unity feedback.

$$sD(s) + (K_d s^2 + K_p s + K_i)N(s) = 0$$

symbol		meaning	SI unit
`num`	in	plant numerator	,
`den`	in	plant denominator	,
`Kp`	in	proportional gain	,
`Ki`	in	integral gain (default 0) (optional)	,
`Kd`	in	derivative gain (default 0) (optional)	,
`char_poly`	out	closed-loop characteristic polynomial	,
`poles`	out	closed-loop poles	,
`max_real_part`	out	largest real part	,
`stable`	out	true when every pole is in the left half plane	,

Applies when: strictly proper plant.

```
$ control-systems solve ctrl_pid_closed_loop "num=[1]" "den=[1, 1]" Kp=3 Ki=2
char_poly = [1.0,4.0,2.0]
poles = [-3.41421356237,-0.585786437627]
max_real_part = -0.5857864376
stable = true
  by: Closed-loop poles of a PID loop  (1 + C(s) G(s) = 0, roots by the Aberth-Ehrlich
iteration)
  Kp = 3
  Ki = 2
  assumes: strictly proper plant
```

What the worked example means: This calculation determines the stability of a control loop by finding the roots of the characteristic polynomial formed from the plant numerator and denominator combined with the proportional, integral, and derivative gains. The resulting closed-loop poles are [-3.41421, -0.585786]. The critical measure for stability is the maximum real part of these poles, which is -0.585786. If this value is negative, the system is stable as indicated by yes; if positive, the system will diverge over time.

Cohen-Coon tuning of a first-order-plus-dead-time process, `ctrl_cohen_coon`

P, PI or PID settings from the Cohen-Coon formulas for K e^{-theta s}/(tau s + 1).

$$\text{PID: } K_p = \frac{\tau}{K\theta}\left(\frac{4}{3} + \frac{\theta}{4\tau}\right),\ T_i = \theta\frac{32 + 6\theta/\tau}{13 + 8\theta/\tau},\ T_d = \frac{4\theta}{11 + 2\theta/\tau}$$

symbol		meaning	SI unit
`K`	in	process gain	,
`tau`	in	time constant in seconds	,
`theta`	in	dead time in seconds	,
`controller`	in	controller type	,
`Kp`	out	proportional gain	,
`Ti`	out	integral time (0 when there is no integral action)	s
`Td`	out	derivative time	s
`Ki`	out	integral gain Kp/Ti	,
`Kd`	out	derivative gain Kp Td	,

Applies when: K, tau, theta > 0.

```
$ control-systems solve ctrl_cohen_coon K=2 tau=10 theta=2 controller=PID
Kp = 3.458333333
Ti = 4.547945205 s
Td = 0.701754386 s
Ki = 0.7604166667
Kd = 2.426900585
  by: Cohen-Coon tuning of a first-order-plus-dead-time process  (Cohen-Coon formulas (r =
theta/tau))
  K = 2
  tau = 10
  theta = 2
  controller = PID
  assumes: K, tau, theta > 0
```

What the worked example means: This calculation determines the recommended PID controller parameters using the Cohen-Coon tuning method. It takes the process gain of 2, the time constant of 10, and the dead time of 2 to compute the proportional gain as 3.45833. The integral time is found to be 4.54795 s seconds, while the derivative time is calculated as 0.701754 s seconds. These values provide a starting point for manual control loops where precise model matching is not required but stability is critical.

PI gains placing the closed-loop poles of a first-order plant, `ctrl_pi_pole_placement`

Kp = (2 zeta wn tau - 1)/K and Ki = wn^2 tau/K, so that K/(tau s + 1) under PI control with unity feedback has the characteristic polynomial s^2 + 2 zeta wn s + wn^2.

$$K_p = \frac{2\zeta\omega_n\tau - 1}{K},\quad K_i = \frac{\omega_n^2\tau}{K}$$

symbol		meaning	SI unit
`K`	in	plant gain	,
`tau`	in	plant time constant in seconds	,
`zeta`	in	desired damping ratio	,

symbol		meaning	SI unit
`wn`	in	desired natural frequency in rad/s	,
`Kp`	out	proportional gain	,
`Ki`	out	integral gain	,

Applies when: tau, zeta, wn > 0; K != 0.

```
$ control-systems solve ctrl_pi_pole_placement K=2 tau=5 zeta=0.7 wn=1
Kp = 3
Ki = 2.5
  by: PI gains placing the closed-loop poles of a first-order plant  (match tau s^2 + (1 + K
Kp) s + K Ki to tau (s^2 + 2 zeta wn s + wn^2))
  K = 2
  tau = 5
  zeta = 0.7
  wn = 1
  assumes: tau, zeta, wn > 0; K != 0
```

What the worked example means: This calculation determines the controller gains required to place the closed-loop poles of a first-order plant at specific locations. Given the plant gain 2, time constant 5, desired damping ratio 0.7, and natural frequency 1, the algorithm computes the proportional gain 3 and integral gain 2.5. These values ensure that the system's characteristic polynomial matches the standard second-order form, thereby achieving the specified dynamic response.

PID: parallel gains to standard form, `ctrl_pid_parallel_to_standard`

The integral time Ti = Kp/Ki and derivative time Td = Kd/Kp of a parallel PID Kp + Ki/s + Kd s.

$$T_i = K_p/K_i, \quad T_d = K_d/K_p$$

symbol		meaning	SI unit
`Kp`	in	proportional gain (> 0)	,
`Ki`	in	integral gain (> 0)	,
`Kd`	in	derivative gain	,
`Ti`	out	integral time	s
`Td`	out	derivative time	s

Applies when: Kp > 0; Ki > 0; Kd >= 0.

```
$ control-systems solve ctrl_pid_parallel_to_standard Kp=6 Ki=3 Kd=3
Ti = 2 s
Td = 0.5 s
  by: PID: parallel gains to standard form  (Ti = Kp/Ki, Td = Kd/Kp)
  Kp = 6
  Ki = 3
  Kd = 3
  assumes: Kp > 0; Ki > 0; Kd >= 0
```

What the worked example means: This calculation converts a parallel PID controller structure into standard form parameters. By dividing the proportional gain by the integral gain, this gives the integral time Ti=2 s. Similarly, dividing the derivative gain by the proportional gain yields the derivative time Td=0.5 s. These time constants

define how aggressively the controller corrects accumulated error and responds to rapid changes in the process variable, respectively.

PID: standard form to parallel gains, `ctrl_pid_standard_to_parallel`

Ki = Kp/Ti and Kd = Kp Td from the standard (ideal ISA) form Kp (1 + 1/(Ti s) + Td s).

$$K_i = K_p/T_i, \quad K_d = K_p T_d$$

symbol		meaning	SI unit
`Kp`	in	proportional gain	,
`Ti`	in	integral (reset) time in seconds	,
`Td`	in	derivative time in seconds	,
`Ki`	out	integral gain	,
`Kd`	out	derivative gain	,

Applies when: Ti > 0; Td >= 0.

```
$ control-systems solve ctrl_pid_standard_to_parallel Kp=4 Ti=2 Td=0.5
Ki = 2
Kd = 2
  by: PID: standard form to parallel gains  (Ki = Kp/Ti, Kd = Kp Td)
  Kp = 4
  Ti = 2
  Td = 0.5
  assumes: Ti > 0; Td >= 0
```

What the worked example means: This calculation converts the standard ideal controller parameters into their parallel gain equivalents. Using a proportional gain of 4, an integral time of 2, and a derivative time of 0.5, the system computes the specific gains for the integral and derivative actions. The resulting integral gain is 2 and the derivative gain is 2. These values represent the effective weighting applied to the accumulated error and the rate of change of the error, respectively, allowing the controller to respond with precise magnitude in each mode.

SIMC (Skogestad) PI tuning, `ctrl_simc_pi`

PI settings Kp = tau/(K (tau_c + theta)), Ti = min(tau, 4 (tau_c + theta)) for K e^{-theta s}/(tau s + 1); the default tau_c equals theta.

$$K_p = \frac{\tau}{K(\tau_c + \theta)}, \; T_i = \min(\tau, 4(\tau_c + \theta))$$

symbol		meaning	SI unit
`K`	in	process gain	,
`tau`	in	time constant in seconds	,
`theta`	in	dead time in seconds	,
`tau_c`	in	desired closed-loop time constant (default theta) (optional)	,
`Kp`	out	proportional gain	,
`Ti`	out	integral time	s

symbol		meaning	SI unit
`Ki`	out	integral gain	,

Applies when: K, tau, theta > 0.

```
$ control-systems solve ctrl_simc_pi K=2 tau=10 theta=1
Kp = 2.5
Ti = 8 s
Ki = 0.3125
  by: SIMC (Skogestad) PI tuning  (Skogestad's SIMC rule)
  K = 2
  tau = 10
  theta = 1
  assumes: K, tau, theta > 0
```

What the worked example means: This calculation applies Skogestad's SIMC rule to determine the Proportional-Integral controller settings for a first-order plus dead-time process. Using the process gain, time constant, dead time, and desired closed-loop time constant, the method computes the proportional gain as 2.5 and the integral time as 8 s. The resulting values represent the recommended tuning parameters that balance speed of response with robustness against model uncertainty.

Ziegler-Nichols tuning from a plant model, `ctrl_zn_from_plant`

The ultimate gain Ku = 1/|G(j w180)| and period Pu = 2 pi/w180 of a plant model, then the Ziegler-Nichols P, PI or PID settings.

$$K_u = \frac{1}{|G(j\omega_{180})|}, \quad P_u = \frac{2\pi}{\omega_{180}}$$

symbol		meaning	SI unit
`num`	in	plant numerator	,
`den`	in	plant denominator	,
`controller`	in	controller type	,
`Kp`	out	proportional gain	,
`Ti`	out	integral time (0 when there is no integral action)	s
`Td`	out	derivative time	s
`Ki`	out	integral gain Kp/Ti	,
`Kd`	out	derivative gain Kp Td	,
`Ku`	out	ultimate gain	,
`Pu`	out	ultimate period	s

Applies when: exactly one phase crossover.

```
$ control-systems solve ctrl_zn_from_plant "num=[1]" "den=[1, 3, 2, 0]" controller=PID
Kp = 3.6
Ti = 2.221441469 s
Td = 0.5553603673 s
Ki = 1.620569369
Kd = 1.999297322
Ku = 6
Pu = 4.442882938 s
  by: Ziegler-Nichols tuning from a plant model  (phase crossover from Im(N(jw) conj D(jw)) =
0; Ziegler-Nichols table)
  controller = PID
  assumes: exactly one phase crossover
```

What the worked example means: This calculation determines controller parameters using the Ziegler-Nichols method. It first identifies the phase crossover frequency to compute the ultimate gain 6 and the ultimate period 4.44288 s. These values are then mapped to specific settings for the selected controller type, resulting in a proportional gain of 3.6, an integral time of 2.22144 s, and a derivative time of 0.55536 s. The final gains Ki and Kd are derived from these primary parameters.

Ziegler-Nichols tuning from the process reaction curve, `ctrl_zn_reaction`

P, PI or PID settings from the open-loop (reaction-curve) Ziegler-Nichols table for a process with gain K, dead time L and time constant T.

$$\text{PID: } K_p = \frac{1.2\,T}{KL},\ T_i = 2L,\ T_d = 0.5L$$

symbol		meaning	SI unit
`K`	in	process gain	,
`L`	in	apparent dead time in seconds	,
`T`	in	apparent time constant in seconds	,
`controller`	in	controller type	,
`Kp`	out	proportional gain	,
`Ti`	out	integral time (0 when there is no integral action)	s
`Td`	out	derivative time	s
`Ki`	out	integral gain Kp/Ti	,
`Kd`	out	derivative gain Kp Td	,

Applies when: K, L, T > 0.

```
$ control-systems solve ctrl_zn_reaction K=2 L=1 T=10 controller=PID
Kp = 6
Ti = 2 s
Td = 0.5 s
Ki = 3
Kd = 3
  by: Ziegler-Nichols tuning from the process reaction curve  (Ziegler-Nichols reaction-curve
table)
  K = 2
  L = 1
  T = 10
  controller = PID
  assumes: K, L, T > 0
```

What the worked example means: This calculation determines PID controller settings using the Ziegler-Nichols reaction curve method. It takes the process gain 2, dead time 1 seconds, and time constant 10 seconds as inputs. The resulting proportional gain is 6, which scales the error signal. The integral time is calculated as 2 s seconds, representing how quickly accumulated errors are corrected, while the derivative time is 0.5 s seconds, indicating the response to the rate of change in the error. These values aim to provide a stable and responsive control loop based on the open-loop step response characteristics.

Ziegler-Nichols tuning from the ultimate gain and period, `ctrl_zn_ultimate`

P, PI or PID settings from the closed-loop (ultimate-cycle) Ziegler-Nichols table: P 0.5 Ku; PI 0.45 Ku, Ti = Pu/1.2; PID 0.6 Ku, Ti = Pu/2, Td = Pu/8.

$$\text{PID: } K_p = 0.6K_u,\ T_i = P_u/2,\ T_d = P_u/8$$

symbol		meaning	SI unit
`Ku`	in	ultimate gain (gain at sustained oscillation)	,
`Pu`	in	ultimate period in seconds	,
`controller`	in	controller type	,
`Kp`	out	proportional gain	,
`Ti`	out	integral time (0 when there is no integral action)	s
`Td`	out	derivative time	s
`Ki`	out	integral gain Kp/Ti	,
`Kd`	out	derivative gain Kp Td	,

Applies when: Ku > 0; Pu > 0.

```
$ control-systems solve ctrl_zn_ultimate Ku=10 Pu=4 controller=PID
Kp = 6
Ti = 2 s
Td = 0.5 s
Ki = 3
Kd = 3
  by: Ziegler-Nichols tuning from the ultimate gain and period  (Ziegler-Nichols ultimate-cycle
table)
  Ku = 10
  Pu = 4
  controller = PID
  assumes: Ku > 0; Pu > 0
```

What the worked example means: This calculation applies the Ziegler-Nichols closed-loop tuning rules to derive controller parameters from the critical gain and oscillation period. By using the specified controller type, the system determines the proportional gain as 6, which directly scales with the ultimate gain 10. The integral time is set to 2 s, derived from the ultimate period 4, while the derivative time is calculated as 0.5 s. These values represent the recommended starting point for manual tuning in a feedback control loop.

Root locus (5)

Root-locus angle of arrival at a complex zero, `ctrl_arrival_angle`

The angle at which the root locus arrives at the complex zero z (upper half plane): 180 + sum of angles from the poles - sum of angles from the other zeros.

$$\theta_a = 180^\circ + \sum \angle(z - p_j) - \sum_{i \neq} \angle(z - z_i)$$

symbol		meaning	SI unit
`num`	in	numerator of G(s) (exactly one complex zero pair)	,
`den`	in	denominator of G(s)	,
`zero`	out	the complex zero (positive imaginary part)	,
`arrival_deg`	out	angle of arrival in degrees, (-180, 180]	,

Applies when: exactly one complex zero pair; same-sign leading coefficients.

```
$ control-systems solve ctrl_arrival_angle "num=[1, 2, 2]" "den=[1, 0]"
zero = {"re":-1.0,"im":1.0}
arrival_deg = -135
  by: Root-locus angle of arrival at a complex zero  (angle criterion at the zero)
  assumes: exactly one complex zero pair; same-sign leading coefficients
```

What the worked example means: This calculation determines the specific angle at which the root locus branches approach the complex zero located at -1 + 1i. The result, given as -135, is derived using the angle criterion, which sums the angles contributed by all system poles and subtracts the angles from other zeros. This value indicates the geometric direction of the trajectory as it terminates at the zero in the upper half of the complex plane.

Root-locus angle of departure from a complex pole, `ctrl_departure_angle`

The angle at which the root locus leaves the complex pole p (upper half plane): 180 + sum of angles from the zeros - sum of angles from the other poles.

$$\theta_d = 180^\circ + \sum \angle(p - z_i) - \sum_{j \neq} \angle(p - p_j)$$

symbol		meaning	SI unit
`num`	in	numerator of G(s)	,
`den`	in	denominator of G(s) (exactly one complex pole pair)	,
`pole`	out	the complex pole (positive imaginary part)	,
`departure_deg`	out	angle of departure in degrees, (-180, 180]	,

Applies when: exactly one complex pole pair; same-sign leading coefficients.

```
$ control-systems solve ctrl_departure_angle "num=[1]" "den=[1, 2, 2, 0]"
pole = {"re":-1.0,"im":1.0}
departure_deg = -45
  by: Root-locus angle of departure from a complex pole  (angle criterion at the pole)
  assumes: exactly one complex pole pair; same-sign leading coefficients
```

What the worked example means: This calculation determines the angle of departure for the root locus branch originating from the complex pole -1 + 1i. The angle criterion sums the angles contributed by all zeros and subtracts the angles contributed by other poles relative to the specific complex pole. The resulting departure angle is -45, indicating the initial direction the closed-loop poles take as gain increases.

Root-locus asymptotes (centroid and angles), `ctrl_locus_asymptotes`

Number of asymptotes n - m, their real-axis intercept (sum of poles - sum of zeros)/(n - m) and their angles (2k+1) 180/(n - m) degrees.

$$\sigma_a = \frac{\sum p_i - \sum z_j}{n - m}, \quad \theta_k = \frac{(2k+1)180^\circ}{n - m}$$

symbol		meaning	SI unit
`num`	in	numerator of the open-loop G(s)	,
`den`	in	denominator of G(s)	,
`n_asymptotes`	out	number of asymptotes	,
`centroid`	out	real-axis intercept	,
`angles_deg`	out	asymptote angles in degrees	,

Applies when: more poles than zeros.

```
$ control-systems solve ctrl_locus_asymptotes "num=[1]" "den=[1, 3, 2, 0]"
n_asymptotes = 3
centroid = -1
angles_deg = [60.0,180.0,300.0]
  by: Root-locus asymptotes (centroid and angles)  (sums of poles and zeros from the
coefficients)
  assumes: more poles than zeros
```

What the worked example means: This calculation determines the behavior of the root locus branches as they extend to infinity. By comparing the number of poles and zeros in the open-loop transfer function, there are 3 asymptotes. These lines originate from a centroid on the real axis located at -1, which is derived from the sum of pole locations minus the sum of zero locations divided by the difference in their counts. The directions of these asymptotes are defined by the angles [60, 180, 300], indicating how the closed-loop poles will move for high gain values.

Root-locus breakaway and break-in points, `ctrl_breakaway_points`

Real points where root-locus branches meet and leave (or enter) the real axis: real roots of N D' - N' D = 0 with a positive gain K = -D/N there.

$$\frac{dK}{ds} = 0 \iff ND' - N'D = 0$$

symbol		meaning	SI unit
`num`	in	numerator of G(s)	,
`den`	in	denominator of G(s)	,
`points`	out	breakaway/break-in points, ascending	,
`gains`	out	gain K at each point	,
`count`	out	number of points	,

Applies when: proper G(s).

```
$ control-systems solve ctrl_breakaway_points "num=[1]" "den=[1, 3, 2, 0]"
points = [-0.42264973081]
gains = [0.38490017946]
count = 1
  by: Root-locus breakaway and break-in points  (dK/ds = 0 with K = -D(s)/N(s))
  assumes: proper G(s)
```

What the worked example means: This calculation determines the specific locations on the real axis where root-locus branches leave or enter by solving for the gain derivative with respect to the complex variable s. The process involves taking the numerator and denominator of the open-loop transfer function, represented as [1] and [1, 3, 2, 0], to find the roots of the characteristic equation derived from the condition that the rate of change of the gain is zero. The resulting positions are listed in ascending order as [-0.42265], while the corresponding proportional gains required to place closed-loop poles at these locations are given as [0.3849]. These values indicate critical transitions in the system's dynamic response as the feedback gain varies.

Root-locus gain and angle at a test point, `ctrl_locus_gain_at_point`

The gain K = 1/|G(s0)| that would place a closed-loop pole at s0 (magnitude criterion) and the angle of G(s0); s0 lies on the root locus for K > 0 when the angle is 180 degrees (angle error 0).

$$K = \frac{1}{|G(s_0)|}, \quad \angle G(s_0) = \pm 180^\circ$$

symbol		meaning	SI unit
`num`	in	numerator of G(s)	,

symbol		meaning	SI unit
`den`	in	denominator of G(s)	,
`s_re`	in	real part of the test point	,
`s_im`	in	imaginary part of the test point	,
`K`	out	gain from the magnitude criterion	,
`angle_deg`	out	angle of G(s0) in degrees	,
`angle_error_deg`	out	180 - |angle|: 0 on the locus	,

Applies when: s0 neither a pole nor a zero.

```
$ control-systems solve ctrl_locus_gain_at_point "num=[1]" "den=[1, 2, 0]" s_re=-1 s_im=1
K = 2
angle_deg = 180
angle_error_deg = 0
  by: Root-locus gain and angle at a test point  (K = |D(s0)|/|N(s0)|)
  s_re = -1
  s_im = 1
  assumes: s0 neither a pole nor a zero
```

What the worked example means: This calculation determines the specific gain K that would place a closed-loop pole at the test point s0, defined by its real part -1 and imaginary part 1. The magnitude criterion yields a gain of 2, while the angle criterion evaluates the phase of the open-loop transfer function at this location. The resulting angle is 180 degrees, leading to an angle error of 0 degrees relative to the ideal 180-degree condition for positive gain. If the angle error is zero, the test point lies exactly on the root locus.

Stability (9)

Continuous Lyapunov equation, `ctrl_lyapunov`

The solution P of A^T P + P A + Q = 0 for a stable A (exact), and whether P is positive definite (with Q > 0 that certifies stability).

$$A^T P + P A + Q = 0$$

symbol		meaning	SI unit
`A`	in	state matrix (stable, n x n)	,
`Q`	in	symmetric matrix (n x n)	,
`P`	out	solution P	,
`positive_definite`	out	true when P is positive definite	,

Applies when: A stable; Q symmetric.

```
$ control-systems solve ctrl_lyapunov "A=[[-1, 0], [0, -2]]" "Q=[[2, 0], [0, 4]]"
P = [[1.0,0.0],[0.0,1.0]]
positive_definite = true
  by: Continuous Lyapunov equation  (Kronecker linear system in exact fractions)
  assumes: A stable; Q symmetric
```

What the worked example means: This calculation solves the continuous Lyapunov equation A^T P + P A + Q = 0 using exact fractions to find the matrix P. The result shows that P equals [[1, 0], [0, 1]]. This solution helps

determine stability, as a positive definite P confirms the system's behavior when Q is positive definite.

Discrete Lyapunov equation, `ctrl_discrete_lyapunov`

The solution P of A^T P A - P + Q = 0 for a Schur-stable A (exact), and whether P is positive definite.

$$A^T P A - P + Q = 0$$

symbol		meaning	SI unit
`A`	in	discrete state matrix (eigenvalues inside the unit circle)	,
`Q`	in	symmetric matrix (n x n)	,
`P`	out	solution P	,
`positive_definite`	out	true when P is positive definite	,

Applies when: A Schur stable; Q symmetric.

```
$ control-systems solve ctrl_discrete_lyapunov "A=[[0.5, 0], [0, 0.5]]" "Q=[[3, 0], [0, 3]]"
P = [[4.0,0.0],[0.0,4.0]]
positive_definite = true
  by: Discrete Lyapunov equation  (Kronecker linear system in exact fractions)
  assumes: A Schur stable; Q symmetric
```

In the worked example, A = [0.5 0; 0 0.5], Q = [3 0; 0 3] go through the equation above and give P = [[4, 0], [0, 4]], positive_definite = True. It applies only when A Schur stable; Q symmetric: outside that it says what is wrong instead of returning a number.

Gain and phase margins, `ctrl_stability_margins`

Gain margin 1/|L(j w_pc)| (ratio and dB) at the phase crossover and phase margin 180 + arg L(j w_gc) at the gain crossover of an open loop L(s).

$$GM = \frac{1}{|L(j\omega_{pc})|}, \quad PM = 180^\circ + \angle L(j\omega_{gc})$$

symbol		meaning	SI unit
`num`	in	numerator of the open loop L(s)	,
`den`	in	denominator of L(s)	,
`gm`	out	gain margin (ratio)	,
`gm_db`	out	gain margin in dB	,
`w_pc`	out	phase-crossover frequency	rad/s
`pm_deg`	out	phase margin in degrees	,
`w_gc`	out	gain-crossover frequency	rad/s

Applies when: exactly one gain crossover and one phase crossover.

```
$ control-systems solve ctrl_stability_margins "num=[10]" "den=[1, 3, 2, 0]"
gm = 0.6
gm_db = -4.436974992
w_pc = 1.414213562 rad/s
pm_deg = -12.99720802
w_gc = 1.802203305 rad/s
  by: Gain and phase margins  (crossovers as positive real roots of |N|^2 - |D|^2 and Im(N(jw)
conj D(jw)), Newton-polished)
  assumes: exactly one gain crossover and one phase crossover
```

What the worked example means: This calculation determines the stability margins for an open-loop transfer function defined by the numerator [10] and denominator [1, 3, 2, 0]. The algorithm identifies specific frequencies where the magnitude or phase conditions are met to find the phase crossover frequency 1.41421 rad/s and the gain crossover frequency 1.8022 rad/s. At the phase crossover, the gain margin 0.6 indicates how much additional gain can be applied before the system becomes unstable. Conversely, at the gain crossover, the phase margin -12.9972 shows how much additional phase lag can be tolerated. These values provide a quantitative measure of robustness against variations in the system parameters.

Jury (Schur-Cohn) stability test for a discrete polynomial, `ctrl_jury_test`

Whether every root of a polynomial in z lies inside the unit circle (Jury/Schur-Cohn table in exact arithmetic), with the necessary-condition values P(1) and (-1)^n P(-1).

$$|a_n| < a_0,\ P(1) > 0,\ (-1)^n P(-1) > 0$$

symbol		meaning	SI unit
`coefficients`	in	polynomial coefficients in z, highest power first	,
`stable`	out	true when every root is inside the unit circle	,
`P1`	out	P(1) (must be > 0)	,
`Pm1`	out	(-1)^n P(-1) (must be > 0)	,

Applies when: the table is not singular.

```
$ control-systems solve ctrl_jury_test "coefficients=[1, -1.2, 0.35]"
stable = true
P1 = 0.15
Pm1 = 2.55
  by: Jury (Schur-Cohn) stability test for a discrete polynomial  (Schur-Cohn reduction q <- q
- (a_n/a_0) q_reversed in exact fractions)
  assumes: the table is not singular
```

What the worked example means: This calculation determines whether all roots of the discrete-time characteristic polynomial lie strictly inside the unit circle in the complex plane. The result depends on the specific coefficients provided as inputs. For stability, the necessary conditions are that the value of the polynomial evaluated at one, represented by 0.15, must be greater than zero, and the alternating sign evaluation at negative one, 2.55, must also be positive. If these conditions hold and the reduction table remains non-singular, the system is deemed stable.

Nyquist criterion: encirclements of -1, `ctrl_nyquist`

For the open loop K G(s) under unity negative feedback: the open-loop right-half-plane poles P, the closed-loop right-half-plane poles Z and the net clockwise encirclements N = Z - P of the point -1 by the Nyquist curve.

$$Z = N + P$$

symbol		meaning	SI unit
`num`	in	numerator of G(s)	,
`den`	in	denominator of G(s)	,
`K`	in	loop gain (default 1) (optional)	,
`P`	out	open-loop poles in the right half plane	,
`Z`	out	closed-loop poles in the right half plane	,
`N`	out	net clockwise encirclements of -1	,
`stable`	out	true when Z = 0	,

Applies when: no open-loop or closed-loop pole on the imaginary axis.

```
$ control-systems solve ctrl_nyquist "num=[1]" "den=[1, -1]" K=2
P = 1
Z = 0
N = -1
stable = true
  by: Nyquist criterion: encirclements of -1  (Z = N + P from the poles of G and of 1 + K G)
  K = 2
  assumes: no open-loop or closed-loop pole on the imaginary axis
```

What the worked example means: This calculation determines the stability of the feedback loop by applying the Nyquist criterion. It computes P, the number of open-loop poles in the right half plane, and N, the net clockwise encirclements of the critical point minus one by the plot of K times G. The result Z, representing the number of closed-loop poles in the right half plane, is found using the relationship Z equals N plus P. With 1 open-loop unstable poles and -1 encirclements, the system ends up with 0 unstable closed-loop poles.

Peak sensitivity Ms and modulus margin, `ctrl_sensitivity_peak`

Ms = max |1/(1 + L(jw))| of a stable unity-feedback loop, its frequency, and the modulus (stability) margin 1/Ms: the smallest distance from the Nyquist curve to -1.

$$M_s = \max_{\omega} \frac{1}{|1 + L(j\omega)|}$$

symbol		meaning	SI unit
`num`	in	numerator of the open loop L(s)	,
`den`	in	denominator of L(s)	,
`Ms`	out	peak sensitivity	,
`Ms_db`	out	peak sensitivity in dB	,
`w_ms`	out	frequency of the peak	rad/s
`sm`	out	modulus margin 1/Ms	,

Applies when: closed loop stable; strictly proper L(s).

```
$ control-systems solve ctrl_sensitivity_peak "num=[2]" "den=[1, 3, 2, 0]"
Ms = 2.312314015
Ms_db = 7.280936228
w_ms = 0.9252900062 rad/s
sm = 0.4324672141
  by: Peak sensitivity Ms and modulus margin  (stationary points of |D(jw)|^2/|D(jw) + N(jw)|^2
(positive real roots of f' g - f g'))
  assumes: closed loop stable; strictly proper L(s)
```

What the worked example means: This calculation determines the maximum magnitude of the sensitivity function for a stable unity feedback loop. The result shows that the peak sensitivity value is 2.31231, reached at the frequency 0.92529 rad/s rad/s. This peak indicates the worst-case amplification of disturbances or reference tracking errors in the system. The associated modulus margin, calculated as the reciprocal of this peak, is 0.432467, representing the shortest distance from the Nyquist plot of the open-loop transfer function to the critical point minus one.

Poles and stability of a characteristic polynomial, `ctrl_pole_stability`

The roots (poles) of a characteristic polynomial and whether the system is asymptotically stable, marginally stable or unstable.

$$\text{stable} \iff \operatorname{Re} p_i < 0 \; \forall i$$

symbol		meaning	SI unit
`den`	in	characteristic polynomial (denominator) coefficients, highest power first	,
`poles`	out	the poles	,
`max_real_part`	out	largest real part of the poles	,
`stability`	out	asymptotically stable, marginally stable or unstable	,

Applies when: degree 1 or more.

```
$ control-systems solve ctrl_pole_stability "den=[1, 6, 11, 6]"
poles = [-3.0,-2.0,-1.0]
max_real_part = -1
stability = asymptotically stable
  by: Poles and stability of a characteristic polynomial  (roots by the Aberth-Ehrlich
iteration; simple poles on the imaginary axis are marginal, repeated ones unstable)
  assumes: degree 1 or more
```

What the worked example means: This calculation determines the stability of a control system by finding the roots of its characteristic polynomial. The inputs are the denominator coefficients, which define the system's dynamics. By applying the Aberth-Ehrlich iteration, this gives the specific pole locations in the complex plane. The result shows that the largest real part among these poles is -1, leading to a classification of asymptotically stable. For asymptotic stability, all poles must have negative real parts; if any pole has a positive real part, the system is unstable.

Range of gain for closed-loop stability (Routh, j-omega crossing), `ctrl_stable_gain_range`

The interval K_min < K < K_max of positive gains for which 1 + K G(s) = 0 has every root in the left half plane, and the frequency at which the root locus crosses the imaginary axis at K_max.

$$D(j\omega) + KN(j\omega) = 0$$

symbol		meaning	SI unit
`num`	in	numerator of G(s)	,
`den`	in	denominator of G(s)	,
`K_min`	out	lower end of the stable range (0 when stable from small gains)	,
`K_max`	out	critical gain (upper end)	,
`w_cross`	out	imaginary-axis crossing frequency at K_max (0 for a crossing at s = 0)	rad/s

Applies when: one bounded interval of stable positive gains.

```
$ control-systems solve ctrl_stable_gain_range "num=[1]" "den=[1, 3, 2, 0]"
K_min = 0
K_max = 6
w_cross = 1.414213562 rad/s
  by: Range of gain for closed-loop stability (Routh, j-omega crossing)  (critical gains from
D(jw) + K N(jw) = 0 and D(0) + K N(0) = 0; each interval tested by an exact Routh array)
  assumes: one bounded interval of stable positive gains
```

What the worked example means: This calculation determines the interval of positive controller gains where the closed-loop system remains stable. It identifies the lower bound 0 and the critical upper bound 6 by analyzing the characteristic equation derived from the open-loop transfer function defined by numerator [1] and denominator [1, 3, 2, 0]. The frequency 1.41421 rad/s indicates where the root locus intersects the imaginary axis, marking the onset of instability if the gain exceeds 6.

Routh-Hurwitz stability test, `ctrl_routh_hurwitz`

The first column of the Routh array (exact arithmetic), the number of right-half-plane roots (its sign changes) and whether the characteristic polynomial is stable.

$$b_i = \frac{a_1 a_{0,i+1} - a_0 a_{1,i+1}}{a_1}$$

symbol		meaning	SI unit
`coefficients`	in	characteristic polynomial coefficients, highest power of s first	,
`first_column`	out	first column of the Routh array	,
`rhp_roots`	out	number of roots in the right half plane	,
`stable`	out	true when every root has a negative real part	,

Applies when: no zero in the first column (special cases are refused).

```
$ control-systems solve ctrl_routh_hurwitz "coefficients=[1, 6, 11, 6]"
first_column = [1.0,6.0,10.0,6.0]
rhp_roots = 0
stable = true
  by: Routh-Hurwitz stability test  (Routh array in exact fractions; sign changes of the first
column)
  assumes: no zero in the first column (special cases are refused)
```

What the worked example means: This calculation determines the stability of a linear time-invariant system by constructing the Routh array from the characteristic polynomial coefficients [1, 6, 11, 6]. The method involves computing intermediate rows using exact fraction arithmetic and examining the signs of the elements in the first column. The output provides the sequence of these first-column entries [1, 6, 10, 6], counts the number of sign changes to identify unstable poles 0, and declares whether the system is stable yes. A stable result means all roots lie in the left half of the complex plane.

State space (14)

Closed-loop poles under state feedback u = -K x, `ctrl_state_feedback_poles`

The characteristic polynomial det(sI - (A - B K)) (exact), the closed-loop poles and stability for a state-feedback gain K.

$$\det(sI - A + BK) = 0$$

symbol		meaning	SI unit
`A`	in	state matrix (n x n)	,
`B`	in	input vector	,
`K`	in	state-feedback gain (row, n entries)	,
`char_poly`	out	closed-loop characteristic polynomial	,
`poles`	out	closed-loop poles	,
`stable`	out	true when every pole is in the left half plane	,

Applies when: single input.

```
$ control-systems solve ctrl_state_feedback_poles "A=[[0, 1], [0, 0]]" "B=[0, 1]" "K=[4, 4]"
char_poly = [1.0,4.0,4.0]
poles = [-2.0,-2.0]
stable = true
  by: Closed-loop poles under state feedback u = -K x  (Faddeev-LeVerrier on A - B K)
  assumes: single input
```

What the worked example means: This calculation determines the dynamic behavior of a system under state feedback control. Given the state matrix A, input vector B, and gain K, the algorithm computes the characteristic polynomial det(sI - (A - BK)) using the Faddeev-LeVerrier method. The resulting polynomial [1, 4, 4] defines the system's eigenvalues, known as closed-loop poles. These poles are located at [-2, -2]. If all poles lie in the left half of the complex plane, the output indicates the system is stable with value yes; otherwise, it is unstable.

Controllability Gramian, `ctrl_ctrb_gramian`

The infinite-horizon controllability Gramian Wc solving A Wc + Wc A^T + B B^T = 0 for a stable A (exact), its determinant and whether (A, B) is controllable (Wc nonsingular).

$$AW_c + W_cA^T + BB^T = 0$$

symbol		meaning	SI unit
`A`	in	state matrix (stable, n x n)	,
`B`	in	input vector	,

symbol		meaning	SI unit
`Wc`	out	controllability Gramian	,
`controllable`	out	true when Wc is nonsingular	,

Applies when: A stable.

```
$ control-systems solve ctrl_ctrb_gramian "A=[[-1, 0], [0, -2]]" "B=[1, 1]"
Wc = [[0.5,0.333333333333],[0.333333333333,0.25]]
controllable = true
  by: Controllability Gramian  (Lyapunov equation in exact fractions)
  assumes: A stable
```

What the worked example means: This calculation determines the infinite-horizon controllability Gramian by solving the Lyapunov equation for a stable state matrix. The resulting Gramian, shown as [[0.5, 0.333333], [0.333333, 0.25]], measures how effectively inputs drive the state variables over time. By checking if this matrix is nonsingular, the pair of matrices is controllable, as indicated by the boolean output yes. A true result confirms that any initial state can be steered to the origin using appropriate control inputs.

Controllability matrix and rank, `ctrl_controllability`

The controllability matrix [B, AB, ..., A^{n-1}B], its exact rank and determinant, and whether (A, B) is controllable.

$$\mathcal{C} = [B\ AB\ \cdots\ A^{n-1}B]$$

symbol		meaning	SI unit
`A`	in	state matrix (n x n)	,
`B`	in	input vector (n entries)	,
`Wc`	out	controllability matrix	,
`rank`	out	rank of the controllability matrix	,
`controllable`	out	true when the rank is n	,
`det_Wc`	out	determinant of the controllability matrix	,

Applies when: single input.

```
$ control-systems solve ctrl_controllability "A=[[0, 1], [-2, -3]]" "B=[0, 1]"
Wc = [[0.0,1.0],[1.0,-3.0]]
rank = 2
controllable = true
det_Wc = -1
  by: Controllability matrix and rank  (exact rank by elimination in fractions)
  assumes: single input
```

What the worked example means: The controllability matrix is built from the state matrix A and input vector B. The resulting matrix Wc has a rank of 2. Since the rank equals the number of states, the pair is controllable, with a determinant of -1.

Controllable canonical form of a transfer function, `ctrl_tf_to_ss`

A, B, C, D in the controllable canonical form (scipy's convention) of a proper transfer function.

$$A = \begin{bmatrix} -a_1 & \cdots & -a_n \\ 1 & & 0 \\ & \ddots & \end{bmatrix},\ B = e_1$$

symbol		meaning	SI unit
`num`	in	numerator coefficients	,
`den`	in	denominator coefficients	,
`A`	out	state matrix	,
`B`	out	input vector	,
`C`	out	output vector	,
`D`	out	feedthrough	,

Applies when: proper G(s).

```
$ control-systems solve ctrl_tf_to_ss "num=[1]" "den=[1, 3, 2]"
A = [[-3.0,-2.0],[1.0,0.0]]
B = [1.0,0.0]
C = [0.0,1.0]
D = 0
  by: Controllable canonical form of a transfer function  (normalise den; first row of A = -
a1..-an; C from num - D den)
  assumes: proper G(s)
```

What the worked example means: This calculation converts a proper transfer function into the controllable canonical form, a standard state-space representation. It takes the numerator coefficients [1] and denominator coefficients [1, 3, 2] to construct the system matrices. The resulting state matrix [[-3, -2], [1, 0]] defines the dynamics, while the input vector [1, 0] determines how the external signal enters the system. The output vector [0, 1] maps the internal states to the observable output, and the feedthrough term 0 represents any direct influence of the input on the output without passing through the states.

DC gain of a state-space model, `ctrl_ss_dc_gain`

The steady-state gain G(0) = D - C A^{-1} B of a state-space model (exact).

$$G(0) = D - CA^{-1}B$$

symbol		meaning	SI unit
`A`	in	state matrix (n x n)	,
`B`	in	input vector	,
`C`	in	output vector	,
`D`	in	feedthrough (default 0) (optional)	,
`dc_gain`	out	DC gain G(0)	,

Applies when: A nonsingular.

```
$ control-systems solve ctrl_ss_dc_gain "A=[[0, 1], [-2, -3]]" "B=[0, 1]" "C=[1, 0]"
dc_gain = 0.5
  by: DC gain of a state-space model  (solve A x = B exactly, then D - C x)
  assumes: A nonsingular
```

What the worked example means: This calculation determines the steady-state gain of a linear time-invariant system described by state-space equations. It uses the formula involving the inverse of the state matrix, multiplied by the input and output vectors, subtracted from the direct feedthrough term. The result, 0.5, represents the ratio of the final output value to the constant input magnitude once transient effects have fully decayed, assuming the system matrix is nonsingular.

Deadbeat state feedback (discrete time), `ctrl_deadbeat_gain`

The gain K that places every eigenvalue of A - B K at z = 0, so the discrete closed loop reaches the origin in at most n steps.

$$K = [0 \cdots 1]\,\mathcal{C}^{-1} A^{n}$$

symbol		meaning	SI unit
`A`	in	discrete state matrix (n x n)	,
`B`	in	input vector	,
`K`	out	deadbeat state-feedback gain	,

Applies when: (A, B) controllable.

```
$ control-systems solve ctrl_deadbeat_gain "A=[[1, 1], [0, 1]]" "B=[0, 1]"
K = [1.0,2.0]
  by: Deadbeat state feedback (discrete time)  (Ackermann's formula with alpha(z) = z^n
(exact))
  assumes: (A, B) controllable
```

What the worked example means: This calculation determines the state-feedback gain [1, 2] for a discrete-time control system. Using Ackermann's formula, it places all closed-loop eigenvalues at zero, ensuring that the system state converges to the origin within n time steps, where n is the dimension of the state matrix [1 1; 0 1]. This result assumes the pair formed by the state matrix [1 1; 0 1] and input vector [0, 1] is controllable.

Eigenvalues (poles) of a state matrix, `ctrl_ss_eigenvalues`

The characteristic polynomial det(sI - A) (exact), the eigenvalues of A and whether the system is asymptotically stable.

$$\det(sI - A) = 0$$

symbol		meaning	SI unit
`A`	in	state matrix (n x n)	,
`eigenvalues`	out	eigenvalues of A	,
`char_poly`	out	characteristic polynomial (monic)	,
`max_real_part`	out	largest real part	,

symbol		meaning	SI unit
`stable`	out	true when every eigenvalue has a negative real part	,

Applies when: A square, at most 6 x 6.

```
$ control-systems solve ctrl_ss_eigenvalues "A=[[0, 1], [-2, -3]]"
eigenvalues = [-2.0,-1.0]
char_poly = [1.0,3.0,2.0]
max_real_part = -1
stable = true
  by: Eigenvalues (poles) of a state matrix  (Faddeev-LeVerrier, then polynomial roots)
  assumes: A square, at most 6 x 6
```

What the worked example means: This calculation determines the poles of the state matrix A by finding the roots of its characteristic polynomial. The resulting eigenvalues, listed as [-2, -1], define the dynamic modes of the system. By examining the real parts of these values, specifically the maximum real part which is -1, asymptotic stability is assessed. If all real parts are negative, the system returns to equilibrium after disturbance, marked here as stable with value yes.

Observability Gramian, `ctrl_obsv_gramian`

The infinite-horizon observability Gramian Wo solving A^T Wo + Wo A + C^T C = 0 for a stable A (exact), its determinant and whether (A, C) is observable.

$$A^T W_o + W_o A + C^T C = 0$$

symbol		meaning	SI unit
`A`	in	state matrix (stable, n x n)	,
`C`	in	output vector	,
`Wo`	out	observability Gramian	,
`observable`	out	true when Wo is nonsingular	,

Applies when: A stable.

```
$ control-systems solve ctrl_obsv_gramian "A=[[-1, 0], [0, -2]]" "C=[1, 1]"
Wo = [[0.5,0.333333333333],[0.333333333333,0.25]]
observable = true
  by: Observability Gramian  (Lyapunov equation in exact fractions)
  assumes: A stable
```

What the worked example means: This calculation solves the Lyapunov equation A transpose times Wo plus Wo times A plus C transpose times C equals zero to find the infinite-horizon observability Gramian. The resulting matrix Wo is [[0.5, 0.333333], [0.333333, 0.25]]. By checking if its determinant is non-zero, this gives if the pair of state and output matrices allows full reconstruction of the internal states from the outputs over time.

Observability matrix and rank, `ctrl_observability`

The observability matrix [C; CA; ...; CA^{n-1}], its exact rank and determinant, and whether (A, C) is observable.

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix}$$

symbol		meaning	SI unit
`A`	in	state matrix (n x n)	,
`C`	in	output vector (n entries)	,
`Wo`	out	observability matrix	,
`rank`	out	rank of the observability matrix	,
`observable`	out	true when the rank is n	,
`det_Wo`	out	determinant of the observability matrix	,

Applies when: single output.

```
$ control-systems solve ctrl_observability "A=[[0, 1], [-2, -3]]" "C=[1, 0]"
Wo = [[1.0,0.0],[0.0,1.0]]
rank = 2
observable = true
det_Wo = 1
  by: Observability matrix and rank  (exact rank by elimination in fractions)
  assumes: single output
```

What the worked example means: This calculation constructs the observability matrix from the state matrix and output vector. The resulting rank of 2 determines if all internal states can be inferred from the output measurements over time. A full rank equal to the system dimension implies that the pair is observable, while a lower rank indicates unobservable modes.

Observer (estimator) gain by pole placement, `ctrl_observer_gain`

The observer gain L that gives A - L C the desired characteristic polynomial (single output), by duality with Ackermann's formula.

$$L = \alpha(A)\,\mathcal{O}^{-1} \begin{bmatrix} 0 \\ \vdots \\ 1 \end{bmatrix}$$

symbol		meaning	SI unit
`A`	in	state matrix (n x n)	,
`C`	in	output vector	,
`alpha`	in	desired observer characteristic polynomial (n+1 coefficients)	,
`L`	out	observer gain (column)	,

Applies when: (A, C) observable.

```
$ control-systems solve ctrl_observer_gain "A=[[0, 1], [0, 0]]" "C=[1, 0]" "alpha=[1, 10, 25]"
L = [10.0,25.0]
  by: Observer (estimator) gain by pole placement  (L^T = Ackermann(A^T, C^T, alpha))
  assumes: (A, C) observable
```

What the worked example means: This calculation determines the observer gain matrix L using Ackermann's formula adapted for estimation. By applying duality between controllability and observability, the method computes the specific column vector L that ensures the error dynamics follow the characteristic polynomial alpha. The resulting gain [10, 25] guarantees that the difference between the true state and the estimated state decays according to the selected pole placement strategy, provided the system remains observable.

Pole placement by Ackermann's formula, `ctrl_ackermann`

The state-feedback gain K that gives A - B K the desired characteristic polynomial alpha(s) (single input), computed exactly.

$$K = [0 \quad \cdots \quad 0 \quad 1]\mathcal{C}^{-1}\alpha(A)$$

symbol		meaning	SI unit
`A`	in	state matrix (n x n)	,
`B`	in	input vector	,
`alpha`	in	desired characteristic polynomial (n+1 coefficients, highest power first)	,
`K`	out	state-feedback gain (row)	,

Applies when: (A, B) controllable.

```
$ control-systems solve ctrl_ackermann "A=[[0, 1], [0, 0]]" "B=[0, 1]" "alpha=[1, 4, 4]"
K = [4.0,4.0]
  by: Pole placement by Ackermann's formula  (K = [0 ... 0 1] Wc^{-1} alpha(A) in exact
fractions)
  assumes: (A, B) controllable
```

What the worked example means: Ackermann's formula computes the single-input state-feedback gain vector [4, 4] from the system matrices [0 1; 0 0] and [0, 1] and the desired characteristic polynomial coefficients [1, 4, 4]. This result means that applying feedback u = -[4, 4]x transforms the open-loop dynamics into a stable or otherwise specified closed-loop system with eigenvalues matching the roots of the desired polynomial. The calculation assumes the pair ([0 1; 0 0], [0, 1]) is completely controllable, ensuring that any set of poles can be assigned.

State response to an initial state and a constant input, `ctrl_ss_response`

x(t) = e^{At} x(0) + (integral of e^{A tau} d tau) B u for x' = A x + B u with a constant input u.

$$x(t) = e^{At}x_0 + \int_0^t e^{A\tau}d\tau\, Bu$$

symbol		meaning	SI unit
`A`	in	state matrix (n x n)	,
`B`	in	input vector	,
`x0`	in	initial state	,

symbol		meaning	SI unit
`u`	in	constant input	,
`t`	in	time in seconds	,
`x`	out	state at time t	,

Applies when: |A| t <= 40; t >= 0.

```
$ control-systems solve ctrl_ss_response "A=[[0, 1], [0, 0]]" "B=[0, 1]" "x0=[0, 0]" u=2 t=3
x = [9.0,6.0]
  by: State response to an initial state and a constant input  (exponential of the augmented
matrix [[A, B u], [0, 0]] t)
  u = 2
  t = 3
  assumes: |A| t <= 40; t >= 0
```

What the worked example means: This calculation determines the system's state vector at a specific time by combining the natural response due to the initial conditions with the forced response caused by a constant input. The result, [9, 6], represents where the system ends up after evolving for 3 seconds under these influences.

State-transition matrix e^{At}, `ctrl_state_transition`

The state-transition matrix Phi(t) = e^{At} that maps x(0) to x(t) for x' = A x.

$$\Phi(t) = e^{At} = \sum_{k} \frac{(At)^k}{k!}$$

symbol		meaning	SI unit
`A`	in	state matrix (n x n)	,
`t`	in	time in seconds	,
`Phi`	out	state-transition matrix	,

Applies when: |A| t <= 40.

```
$ control-systems solve ctrl_state_transition "A=[[0, 1], [0, 0]]" t=2
Phi = [[1.0,2.0],[0.0,1.0]]
  by: State-transition matrix e^{At}  (scaling and squaring with a Taylor series)
  t = 2
  assumes: |A| t <= 40
```

What the worked example means: This calculation determines the state-transition matrix [[1, 2], [0, 1]], which describes how the system evolves from its starting condition. It is derived from the state matrix [0 1; 0 0] and the elapsed time 2. The result represents the linear operator that transforms the initial state vector into the state vector at time 2, effectively capturing the dynamic behavior of the system over that interval.

Transfer function of a state-space model, `ctrl_ss_to_tf`

G(s) = C (sI - A)^{-1} B + D of a single-input single-output state-space model, computed exactly (Faddeev-LeVerrier).

$$G(s) = C(sI - A)^{-1}B + D = \frac{C\,\mathrm{adj}(sI - A)\,B}{\det(sI - A)} + D$$

symbol		meaning	SI unit
`A`	in	state matrix (n x n)	,
`B`	in	input vector (column, n entries)	,
`C`	in	output vector (row, n entries)	,
`D`	in	feedthrough (default 0) (optional)	,
`num`	out	numerator (n+1 coefficients, highest power first)	,
`den`	out	characteristic polynomial (monic)	,

Applies when: A square, at most 6 x 6.

```
$ control-systems solve ctrl_ss_to_tf "A=[[0, 1], [-2, -3]]" "B=[0, 1]" "C=[1, 0]"
num = [0.0,0.0,1.0]
den = [1.0,3.0,2.0]
  by: Transfer function of a state-space model  (Faddeev-LeVerrier in exact fractions)
  assumes: A square, at most 6 x 6
```

What the worked example means: This algorithm computes the transfer function of a single-input single-output state-space model. It derives the numerator coefficients [0, 0, 1] and the monic characteristic polynomial coefficients [1, 3, 2] from the system matrices A, B, C, and D. The result represents the ratio of output to input in the frequency domain.

Steady-state error (1)

System type, static error constant and steady-state error, `ctrl_error_constants`

For unity feedback around G(s): the system type N (poles at the origin), the static error constant lim s^N G(s) (Kp, Kv or Ka) and the steady-state error to a step, ramp or parabola of size A.

$$e_{ss} = \frac{A}{1 + K_p},\ \frac{A}{K_v},\ \frac{A}{K_a}$$

symbol		meaning	SI unit
`num`	in	numerator of the open-loop G(s)	,
`den`	in	denominator of G(s)	,
`input`	in	test input	,
`A`	in	input size (default 1) (optional)	,
`system_type`	out	system type N	,
`K_static`	out	static error constant of type N (Kp if 0, Kv if 1, Ka if 2)	,
`ess`	out	steady-state error	,

Applies when: closed loop stable; proper G(s); type at least the input's order.

```
$ control-systems solve ctrl_error_constants "num=[10]" "den=[1, 3, 2]" input=step
system_type = 0
K_static = 5
ess = 0.1666666667
  by: System type, static error constant and steady-state error  (ess = A/(1 + Kp) (step, type
0), A/Kv (ramp, type 1), A/Ka (parabola, type 2), 0 for a higher type)
  input = step
  assumes: closed loop stable; proper G(s); type at least the input's order
```

What the worked example means: The system type is 0, with a static error constant of 5 and a steady-state error of 0.166667.

Time delay (1)

Pade approximation of a time delay, `ctrl_pade`

The (n, n) Pade approximation of e^{-theta s} as a rational transfer function (denominator's leading coefficient 1).

$$e^{-\theta s} \approx \frac{\sum_k c_k (-\theta s)^k}{\sum_k c_k (\theta s)^k}$$

symbol		meaning	SI unit
`theta`	in	time delay in seconds	,
`n`	in	approximation order (1 to 6)	,
`num`	out	numerator coefficients	,
`den`	out	denominator coefficients (leading 1)	,

Applies when: theta > 0; 1 <= n <= 6.

```
$ control-systems solve ctrl_pade theta=2 n=1
num = [-1.0,1.0]
den = [1.0,1.0]
  by: Pade approximation of a time delay  (Pade coefficients c_k = (2n-k)! n! / ((2n)! k! (n-
k)!))
  theta = 2
  assumes: theta > 0; 1 <= n <= 6
```

What the worked example means: This calculation computes the rational transfer function that approximates the exponential time delay term using the Pade method. Given a specific delay value 2 and an approximation order 1, the algorithm determines the coefficients for the numerator polynomial, reported as [-1, 1], and the denominator polynomial, reported as [1, 1]. The resulting ratio of these polynomials serves as a finite-dimensional substitute for the infinite-dimensional delay operator, allowing standard control system analysis techniques to be applied to systems with transport lags.

Time response (17)

Damping ratio from percent overshoot, `ctrl_damping_from_overshoot`

The damping ratio zeta = -ln(OS)/sqrt(pi^2 + ln^2 OS) that gives a stated overshoot.

$$\zeta = \frac{-\ln(\%OS/100)}{\sqrt{\pi^2 + \ln^2(\%OS/100)}}$$

symbol		meaning	SI unit
`OS`	in	overshoot (write 20% or 0.2)	,
`zeta`	out	damping ratio	,
`pole_angle_deg`	out	pole angle from the negative real axis, degrees	,

Applies when: 0 < OS < 100 %.

```
$ control-systems solve ctrl_damping_from_overshoot OS=0.2
zeta = 0.4559498108
pole_angle_deg = 62.87393661
  by: Damping ratio from percent overshoot  (invert OS = exp(-zeta pi / sqrt(1 - zeta^2)))
  OS = 0.2
  assumes: 0 < OS < 100 %
```

What the worked example means: This calculation determines the damping ratio and the corresponding pole angle from a specified percent overshoot. By inverting the relationship between overshoot and damping, an overshoot of 0.2 yields a damping ratio of 0.45595. This value corresponds to poles located at an angle of 62.8739 degrees from the negative real axis in the complex plane, which characterizes the system's transient response behavior.

Exact settling time of a second-order step response, `ctrl_true_settling_time`

The last time at which the unit step response of wn^2/(s^2 + 2 zeta wn s + wn^2) leaves the band |y - 1| = band (not the envelope estimate), and which extremum precedes it.

$$T_s = \max\{t : |y(t) - 1| = \text{band}\}$$

symbol		meaning	SI unit
`zeta`	in	damping ratio (0 < zeta < 1)	,
`wn`	in	natural frequency in rad/s	,
`band`	in	settling band (default 2%) (optional)	,
`Ts`	out	exact settling time	s
`last_extremum`	out	index k of the last extremum outside the band (0 = before the first peak)	,

Applies when: 0 < zeta < 1; 0 < band < 50 %.

```
$ control-systems solve ctrl_true_settling_time zeta=0.5 wn=1
Ts = 8.076348974 s
last_extremum = 2
  by: Exact settling time of a second-order step response  (extrema at k pi/wd; the last band
crossing lies between the last extremum outside the band and the next zero of y - 1
(bisection))
  zeta = 0.5
  wn = 1
  assumes: 0 < zeta < 1; 0 < band < 50 %
```

What the worked example means: This calculation finds the precise moment when the unit step response of a second-order system stops oscillating within a specified tolerance band. The result, 8.07635 s, represents the latest time any part of the response curve crosses the boundary defined by the absolute difference between the output and one being equal to the band width. This specific value is derived from the damping ratio 0.5 and natural frequency 1, ensuring that after this time, the response remains strictly inside the acceptable error range. The index 2 identifies which peak or trough immediately precedes this final crossing event.

Final value theorem, `ctrl_final_value_theorem`

lim_{t->inf} y(t) = lim_{s->0} s Y(s) for Y(s) = num/den, after checking that s Y(s) has every pole in the open left half plane.

$$y(\infty) = \lim_{s \to 0} sY(s)$$

symbol		meaning	SI unit
`num`	in	numerator of Y(s)	,
`den`	in	denominator of Y(s)	,
`final_value`	out	final value of y(t)	,

Applies when: at most one pole at the origin; all other poles in the open left half plane.

```
$ control-systems solve ctrl_final_value_theorem "num=[10]" "den=[1, 3, 2, 0]"
final_value = 5
  by: Final value theorem  (cancel factors s, then evaluate s Y(s) at s = 0)
  assumes: at most one pole at the origin; all other poles in the open left half plane
```

What the worked example means: This calculation determines the steady-state value that the system output approaches as time goes to infinity. It uses the final value theorem, which states that this limit equals the limit of s multiplied by the Laplace transform of the output as s approaches zero. The process involves canceling any common factors of s between the numerator and denominator before substituting s equals zero into the remaining expression. The result, 5, represents the long-term behavior of the signal y(t), assuming the system is stable enough for the theorem to apply.

Initial value theorem (value and slope at t = 0+), `ctrl_initial_value_theorem`

y(0+) = lim_{s->inf} s Y(s) and y'(0+) = lim_{s->inf} s (s Y(s) - y(0+)) for a strictly proper Y(s).

$$y(0^+) = \lim_{s \to \infty} sY(s)$$

symbol		meaning	SI unit
`num`	in	numerator of Y(s)	,
`den`	in	denominator of Y(s)	,
`y0`	out	initial value y(0+)	,
`dy0`	out	initial slope y'(0+)	,

Applies when: Y(s) strictly proper.

```
$ control-systems solve ctrl_initial_value_theorem "num=[2, 3]" "den=[1, 5, 6]"
y0 = 2
dy0 = -7
  by: Initial value theorem (value and slope at t = 0+)  (long division of num by den in powers
of 1/s)
  assumes: Y(s) strictly proper
```

What the worked example means: This calculation determines the starting point of the system's response by finding the limit as s approaches infinity. The initial value, represented as 2, indicates where the output begins at time zero plus. The initial slope, denoted by -7, describes how rapidly the output changes immediately after the start. These values are derived from the numerator [2, 3] and denominator [1, 5, 6] of the transfer function using polynomial division.

Mass-spring-damper natural frequency and damping ratio, `ctrl_mass_spring_damper`

Natural frequency sqrt(k/m), damping ratio c/(2 sqrt(k m)), damped frequency and static compliance 1/k of m x'' + c x' + k x = F.

$$\omega_n = \sqrt{k/m}, \quad \zeta = \frac{c}{2\sqrt{km}}$$

symbol		meaning	SI unit
`m`	in	mass in kg	,
`c`	in	damping coefficient in N*s/m	,
`k`	in	spring stiffness in N/m	,
`wn`	out	undamped natural frequency	rad/s
`zeta`	out	damping ratio	,
`wd`	out	damped natural frequency (0 when not underdamped)	rad/s
`dc_gain`	out	static gain x/F = 1/k in m/N	,
`damping`	out	damping class	,

Applies when: m > 0; c >= 0; k > 0.

```
$ control-systems solve ctrl_mass_spring_damper m=1 c=2 k=4
wn = 2 rad/s
zeta = 0.5
wd = 1.732050808 rad/s
dc_gain = 0.25
damping = underdamped
  by: Mass-spring-damper natural frequency and damping ratio  (G(s) = 1/(m s^2 + c s + k))
  m = 1
  c = 2
  k = 4
  assumes: m > 0; c >= 0; k > 0
```

What the worked example means: This calculation determines the dynamic characteristics of a linear mass-spring-damper system defined by mass 1, damping coefficient 2, and spring stiffness 4. It computes the undamped natural frequency 2 rad/s in radians per second, which represents the rate at which the system would oscillate if there were no friction. The damping ratio 0.5 quantifies how quickly oscillations decay relative to the

critical damping threshold. Additionally, it provides the damped natural frequency 1.73205 rad/s, the static compliance gain 0.25, and classifies the system's behavior based on the value of 0.5.

Natural frequency and damping ratio of a second-order denominator, `ctrl_second_order_params`

wn, zeta, the decay rate sigma = zeta wn, the damped frequency wd and the DC gain of K/(a2 s^2 + a1 s + a0).

$$s^2 + 2\zeta\omega_n s + \omega_n^2$$

symbol		meaning	SI unit
`den`	in	denominator [a2, a1, a0]	,
`K`	in	numerator constant (default: unit DC gain) (optional)	,
`wn`	out	natural frequency	rad/s
`zeta`	out	damping ratio	,
`sigma`	out	decay rate zeta wn (1/s)	,
`wd`	out	damped frequency (0 unless underdamped)	rad/s
`damping`	out	damping class	,
`dc_gain`	out	DC gain K/a0	,

Applies when: a0/a2 > 0; a1/a2 >= 0.

```
$ control-systems solve ctrl_second_order_params "den=[1, 4, 25]"
wn = 5 rad/s
zeta = 0.4
sigma = 2
wd = 4.582575695 rad/s
damping = underdamped
dc_gain = 1
  by: Natural frequency and damping ratio of a second-order denominator  (compare with s^2 + 2
zeta wn s + wn^2)
  assumes: a0/a2 > 0; a1/a2 >= 0
```

What the worked example means: This calculation identifies the dynamic characteristics of a second-order system by comparing its denominator to the standard form. The resulting natural frequency is 5 rad/s rad/s, while the damping ratio is 0.4. These values determine the transient response speed and oscillation behavior. The exponential decay rate is calculated as 2, indicating how quickly disturbances diminish. If the system is underdamped, the damped frequency is 4.58258 rad/s rad/s; otherwise, it remains zero. The steady-state output for a constant input is scaled by the DC gain of 1.

Peak time of a second-order step response, `ctrl_peak_time`

Peak time Tp = pi/wd and damped frequency wd = wn sqrt(1 - zeta^2) of wn^2/(s^2 + 2 zeta wn s + wn^2).

$$T_p = \frac{\pi}{\omega_n \sqrt{1 - \zeta^2}}$$

symbol		meaning	SI unit
`zeta`	in	damping ratio (0 <= zeta < 1)	,
`wn`	in	natural frequency in rad/s	,

symbol		meaning	SI unit
`Tp`	out	peak time	s
`wd`	out	damped frequency	rad/s

Applies when: 0 <= zeta < 1; wn > 0.

```
$ control-systems solve ctrl_peak_time zeta=0.6 wn=5
Tp = 0.7853981634 s
wd = 4 rad/s
  by: Peak time of a second-order step response  (first zero of the impulse response)
  zeta = 0.6
  wn = 5
  assumes: 0 <= zeta < 1; wn > 0
```

What the worked example means: The peak time, which is the moment the step response reaches its first maximum, is calculated using the damping ratio of 0.6 and the natural frequency of 5. This computation relies on the damped frequency of 4 rad/s. The resulting peak time is 0.785398 s. A higher damping ratio pushes this peak later in time, while a higher natural frequency brings it sooner.

Percent overshoot from the damping ratio, `ctrl_percent_overshoot`

Percent overshoot 100 e^{-zeta pi/sqrt(1 - zeta^2)} of the step response of a standard second-order system, and the pole angle acos(zeta).

$$\%OS = 100\, e^{-\zeta\pi/\sqrt{1-\zeta^2}}$$

symbol		meaning	SI unit
`zeta`	in	damping ratio (0 <= zeta < 1)	,
`PO`	out	percent overshoot (%)	,
`Mp`	out	overshoot as a fraction of the final value	,
`pole_angle_deg`	out	angle of the poles from the negative real axis, degrees	,

Applies when: 0 <= zeta < 1.

```
$ control-systems solve ctrl_percent_overshoot zeta=0.5
PO = 16.30335348
Mp = 0.1630335348
pole_angle_deg = 60
  by: Percent overshoot from the damping ratio  (peak of the step response at t = pi/wd)
  zeta = 0.5
  assumes: 0 <= zeta < 1
```

What the worked example means: This calculation determines the transient response characteristics of a standard second-order system based on its damping ratio. Given an input damping ratio of 0.5, the model computes the peak overshoot as a fraction of the final value, which is 0.163034, and converts this to a percentage of 16.3034. It also calculates the geometric angle of the complex poles relative to the negative real axis, resulting in 60. These values indicate how oscillatory the step response will be; a lower damping ratio leads to higher overshoot and a larger pole angle.

Proportional control of a first-order plant, `ctrl_closed_loop_first_order_p`

Closed-loop DC gain K Kp/(1 + K Kp), time constant tau/(1 + K Kp), steady-state error 1/(1 + K Kp) to a unit step and the closed-loop pole, for K/(tau s + 1) under unity feedback with gain Kp.

$$T(s) = \frac{K K_p}{\tau s + 1 + K K_p}$$

symbol		meaning	SI unit
`K`	in	plant DC gain	,
`tau`	in	plant time constant in seconds	,
`Kp`	in	proportional gain	,
`dc_gain_cl`	out	closed-loop DC gain	,
`tau_cl`	out	closed-loop time constant	s
`ess`	out	steady-state error to a unit step	,
`pole`	out	closed-loop pole (1/s)	,

Applies when: tau > 0; 1 + K Kp > 0.

```
$ control-systems solve ctrl_closed_loop_first_order_p K=2 tau=10 Kp=4.5
dc_gain_cl = 0.9
tau_cl = 1 s
ess = 0.1
pole = -1
  by: Proportional control of a first-order plant  (T(s) = K Kp/(tau s + 1 + K Kp))
  K = 2
  tau = 10
  Kp = 4.5
  assumes: tau > 0; 1 + K Kp > 0
```

What the worked example means: This calculation determines the performance of a proportional controller acting on a first-order plant. Given the plant parameters and controller gain, it computes the closed-loop DC gain as 0.9, the effective time constant as 1 s, and the steady-state error to a unit step as 0.1. These values indicate how quickly the system responds and how accurately it tracks the target value in steady state.

Rise time of an underdamped second-order system, `ctrl_rise_time_second`

The 0-100 % rise time (pi - acos zeta)/wd and the 10-90 % rise time (found exactly by bisection on the step response).

$$T_{r,0-100} = \frac{\pi - \cos^{-1}\zeta}{\omega_d}$$

symbol		meaning	SI unit
`zeta`	in	damping ratio (0 < zeta < 1)	,
`wn`	in	natural frequency in rad/s	,
`Tr_0_100`	out	0-100 % rise time	s
`Tr_10_90`	out	10-90 % rise time	s
`t10`	out	time to 10 %	s
`t90`	out	time to 90 %	s

Applies when: 0 < zeta < 1; wn > 0.

```
$ control-systems solve ctrl_rise_time_second zeta=0.5 wn=1
Tr_0_100 = 2.418399152 s
Tr_10_90 = 1.637572947 s
t10 = 0.4882292958 s
t90 = 2.125802243 s
  by: Rise time of an underdamped second-order system  (closed form (0-100 %); bisection on
y(t) (10-90 %))
  zeta = 0.5
  wn = 1
  assumes: 0 < zeta < 1; wn > 0
```

What the worked example means: This calculation determines the rise time of an underdamped second-order system, specifically the time required for the step response to go from zero to one hundred percent of its final value. The result depends on the damping ratio and the natural frequency, yielding a rise time of 2.4184 s seconds. This metric indicates how quickly the system responds to changes in input, with smaller values representing faster transient responses.

Second-order specifications from the pole location, `ctrl_second_order_from_poles`

From poles at -sigma +/- j wd: wn, zeta, peak time pi/wd, 2 % settling time 4/sigma and percent overshoot 100 e^{-sigma pi/wd}.

$$\omega_n = \sqrt{\sigma^2 + \omega_d^2}, \quad \zeta = \sigma/\omega_n, \quad \%OS = 100e^{-\sigma\pi/\omega_d}$$

symbol		meaning	SI unit
`sigma`	in	decay rate: the poles' real part is -sigma (sigma > 0)	,
`wd`	in	imaginary part of the poles (rad/s)	,
`wn`	out	natural frequency	rad/s
`zeta`	out	damping ratio	,
`Tp`	out	peak time	s
`Ts`	out	2 % settling time 4/sigma	s
`PO`	out	percent overshoot (%)	,

Applies when: sigma > 0; wd > 0.

```
$ control-systems solve ctrl_second_order_from_poles sigma=3 wd=4
wn = 5 rad/s
zeta = 0.6
Tp = 0.7853981634 s
Ts = 1.333333333 s
PO = 9.478022484
  by: Second-order specifications from the pole location  (wn = |p|, zeta = sigma/wn)
  sigma = 3
  wd = 4
  assumes: sigma > 0; wd > 0
```

What the worked example means: This calculation determines the time-domain performance specifications for a second-order system based on its pole locations. Given a decay rate of 3 and an oscillation frequency of 4 rad/s, the natural frequency is computed as 5 rad/s rad/s. The resulting damping ratio is 0.6, which dictates the system's

oscillatory behavior. Key metrics include a peak time of 0.785398 s seconds, a two-percent settling time of 1.33333 s seconds, and a percent overshoot of 9.47802.

Settling time of a second-order system (rule and envelope), `ctrl_settling_time`

Settling time by the rule 4/(zeta wn) (2 % band) or 3/(zeta wn) (5 %), and the time at which the decay envelope enters the band.

$$T_s \approx \frac{4}{\zeta \omega_n}\ (2\%), \qquad T_s \approx \frac{3}{\zeta \omega_n}\ (5\%)$$

symbol		meaning	SI unit
`zeta`	in	damping ratio (0 < zeta < 1)	,
`wn`	in	natural frequency in rad/s	,
`band`	in	settling band: 2% (default) or 5% (optional)	,
`Ts`	out	settling time by the rule of thumb	s
`Ts_envelope`	out	time at which e^{-zeta wn t}/sqrt(1-zeta^2) equals the band	s
`time_constant`	out	envelope time constant 1/(zeta wn)	s

Applies when: 0 < zeta < 1; band 2 % or 5 %.

```
$ control-systems solve ctrl_settling_time zeta=0.5 wn=4
Ts = 2 s
Ts_envelope = 2.027932021 s
time_constant = 0.5 s
  by: Settling time of a second-order system (rule and envelope)  (Ts = 4/(zeta wn) or 3/(zeta
wn))
  zeta = 0.5
  wn = 4
  assumes: 0 < zeta < 1; band 2 % or 5 %
```

What the worked example means: This calculation determines the settling time for a second-order system using the standard rule of thumb. Based on the provided damping ratio and natural frequency, the estimated time for the response to remain within the specified error band is 2 s seconds. The underlying exponential decay envelope reaches this threshold at 2.02793 s seconds, governed by a time constant of 0.5 s. This result indicates how quickly the transient oscillations damp out relative to the steady-state value.

Step response of a first-order system, `ctrl_first_order_step`

The output y(t) = K A (1 - e^{-t/tau}) of K/(tau s + 1) at time t after a step of size A, and the fraction of the final value reached.

$$y(t) = KA\left(1 - e^{-t/\tau}\right)$$

symbol		meaning	SI unit
`K`	in	steady-state (DC) gain of the system	,
`tau`	in	time constant in seconds	,
`t`	in	time after the step in seconds	,

symbol		meaning	SI unit
`A`	in	step size (default 1) (optional)	,
`y`	out	output at time t	,
`fraction`	out	fraction of the final value reached, 1 - e^{-t/tau}	,
`y_final`	out	final value K A	,

Applies when: tau > 0; t >= 0; K and A not zero.

```
$ control-systems solve ctrl_first_order_step K=2 tau=5 t=5
y = 1.264241118
fraction = 0.6321205588
y_final = 2
  by: Step response of a first-order system  (y = K A (1 - exp(-t/tau)))
  K = 2
  tau = 5
  t = 5
  assumes: tau > 0; t >= 0; K and A not zero
```

What the worked example means: This calculation determines the system response y(t) for a first-order model with gain K and time constant tau. Given a step input of magnitude A, the current output is computed as 1.26424. This value represents 0.632121 of the total possible change, which equals the steady-state value of 2. The result indicates how close the system has come to its final target at the specified time t.

Step response of a first-order-plus-dead-time process, `ctrl_fopdt_step`

y(t) of K e^{-theta s}/(tau s + 1) after a step of size A: 0 before the dead time, K A (1 - e^{-(t - theta)/tau}) after it.

$$y(t) = KA\left(1 - e^{-(t-\theta)/\tau}\right) u(t-\theta)$$

symbol		meaning	SI unit
`K`	in	process gain	,
`tau`	in	time constant in seconds	,
`theta`	in	dead time in seconds	,
`t`	in	time in seconds	,
`A`	in	step size (default 1) (optional)	,
`y`	out	output at time t	,

Applies when: tau > 0; theta >= 0; t >= 0.

```
$ control-systems solve ctrl_fopdt_step K=2 tau=5 theta=1 t=6
y = 1.264241118
  by: Step response of a first-order-plus-dead-time process  (delayed first-order response)
  K = 2
  tau = 5
  theta = 1
  t = 6
  assumes: tau > 0; theta >= 0; t >= 0
```

What the worked example means: This calculation determines the process variable response to a step input. It uses the first order plus dead time model, where the output remains zero until the dead time elapses. After that

delay, the value rises exponentially toward the final steady state determined by the product of the process gain and the step size. The current result shows an output of 1.26424 at the specified time.

Step response of a standard second-order system, `ctrl_second_order_step`

y(t) of K wn^2/(s^2 + 2 zeta wn s + wn^2) after a unit step, for underdamped, critically damped and overdamped systems, and the remaining error K - y.

$$y(t) = 1 - e^{-\zeta\omega_n t}\left(\cos\omega_d t + \frac{\zeta}{\sqrt{1-\zeta^2}}\sin\omega_d t\right)$$

symbol		meaning	SI unit
`zeta`	in	damping ratio (>= 0)	,
`wn`	in	natural frequency in rad/s	,
`t`	in	time in seconds	,
`K`	in	DC gain (default 1) (optional)	,
`y`	out	output at time t	,
`error`	out	K - y(t)	,

Applies when: zeta >= 0; wn > 0; t >= 0.

```
$ control-systems solve ctrl_second_order_step zeta=0.5 wn=2 t=1
y = 0.8494256349
error = 0.1505743651
  by: Step response of a standard second-order system  (closed-form step response for each
damping case)
  zeta = 0.5
  wn = 2
  t = 1
  assumes: zeta >= 0; wn > 0; t >= 0
```

What the worked example means: This calculation determines the time-domain output of a standard second-order system following a unit step input. Using the damping ratio, natural frequency, and time provided, it computes the specific value of the response at that instant. The result shows how close the system has come to its final steady-state target, represented here as 0.849426, while also quantifying the remaining deviation from the desired level as 0.150574.

Step response of a transfer function at a given time, `ctrl_step_response`

y(t) of any proper G(s) after a step of size A, computed exactly through the state-space matrix exponential.

$$y(t) = C\int_0^t e^{A\tau}d\tau\, B\, A_u + DA_u$$

symbol		meaning	SI unit
`num`	in	numerator	,
`den`	in	denominator	,
`t`	in	time in seconds	,

symbol		meaning	SI unit
`A`	in	step size (default 1) (optional)	,
`y`	out	output at time t	,

Applies when: proper G(s); t >= 0.

```
$ control-systems solve ctrl_step_response "num=[1]" "den=[1, 1]" t=1
y = 0.6321205588
  by: Step response of a transfer function at a given time  (controllable canonical form;
exponential of [[A, B], [0, 0]] t)
  t = 1
  assumes: proper G(s); t >= 0
```

In the worked example, num = [1], den = [1, 1], t = 1 go through the equation above and give y = 0.6321205588. It applies only when proper G(s); t >= 0: outside that it says what is wrong instead of returning a number.

Time and frequency specifications of a first-order system, `ctrl_first_order_specs`

Time constant, 10-90 % rise time tau ln 9, 2 % and 5 % settling times tau ln 50 and tau ln 20, bandwidth 1/tau and DC gain of K/(tau s + 1).

$$t_r = \tau \ln 9, \quad t_{s,2\%} = \tau \ln 50, \quad \omega_B = 1/\tau$$

symbol		meaning	SI unit
`K`	in	steady-state (DC) gain	,
`tau`	in	time constant in seconds	,
`time_constant`	out	time constant (63.2 % of the final value)	s
`rise_time`	out	10-90 % rise time	s
`settling_time_2`	out	2 % settling time	s
`settling_time_5`	out	5 % settling time	s
`bandwidth`	out	-3 dB bandwidth	rad/s
`dc_gain`	out	DC gain	,

Applies when: tau > 0.

```
$ control-systems solve ctrl_first_order_specs K=3 tau=2
time_constant = 2 s
rise_time = 4.394449155 s
settling_time_2 = 7.824046011 s
settling_time_5 = 5.991464547 s
bandwidth = 0.5 rad/s
dc_gain = 3
  by: Time and frequency specifications of a first-order system  (exact crossing times of 1 -
exp(-t/tau))
  K = 3
  tau = 2
  assumes: tau > 0
```

What the worked example means: This calculation determines key performance metrics for a first-order linear system defined by its steady-state gain 3 and time constant 2. The time constant itself, reported as 2 s, represents

the time required to reach approximately 63.2 percent of the final value. From this base parameter, the exact crossing times are derived: the 10-90 percent rise time is calculated as 4.39445 s, while the 2 percent and 5 percent settling times are 7.82405 s and 5.99146 s respectively. In the frequency domain, the -3 dB bandwidth is found to be 0.5 rad/s rad/s, indicating the range of frequencies the system can effectively track, while the DC gain remains 3. These values collectively describe how quickly the system responds to changes and how much high-frequency noise it filters out.

Transfer functions (6)

Closed-loop transfer function of a feedback loop, `ctrl_feedback_connection`

G/(1 + G H) for negative feedback (or G/(1 - G H) for positive feedback): numerator N_G D_H and denominator D_G D_H +/- N_G N_H.

$$T(s) = \frac{G}{1 + GH} = \frac{N_G D_H}{D_G D_H + N_G N_H}$$

symbol		meaning	SI unit
`num_g`	in	numerator of the forward path G	,
`den_g`	in	denominator of G	,
`num_h`	in	numerator of the feedback path H (default 1) (optional)	,
`den_h`	in	denominator of H (default 1) (optional)	,
`sign`	in	feedback sign (default negative) (optional)	,
`num`	out	closed-loop numerator	,
`den`	out	closed-loop denominator	,

```
$ control-systems solve ctrl_feedback_connection "num_g=[10]" "den_g=[1, 2, 0]"
num = [10.0]
den = [1.0,2.0,10.0]
  by: Closed-loop transfer function of a feedback loop  (T = N_G D_H / (D_G D_H + N_G N_H))
```

What the worked example means: This calculation determines the overall system response by combining the forward path and feedback path polynomials. The resulting closed-loop numerator is [10], derived from the forward path numerator multiplied by the feedback path denominator. The closed-loop denominator is [1, 2, 10], which includes the product of the individual denominators plus or minus the product of the numerators, depending on the feedback sign.

DC gain and final value of a step response, `ctrl_dc_gain`

The DC gain G(0) of a stable transfer function and the final value G(0) A of its response to a step of size A.

$$y(\infty) = A\,G(0)$$

symbol		meaning	SI unit
`num`	in	numerator coefficients, highest power of s first	,
`den`	in	denominator coefficients, highest power of s first	,
`A`	in	step size (default 1) (optional)	,
`dc_gain`	out	DC gain G(0)	,

symbol		meaning	SI unit
`final_value`	out	final value of the step response	,

Applies when: proper G(s); every pole in the left half plane.

```
$ control-systems solve ctrl_dc_gain "num=[10]" "den=[1, 3, 2]"
dc_gain = 5
final_value = 5
  by: DC gain and final value of a step response  (G(0) = num(0)/den(0) after a stability check
of the poles)
  assumes: proper G(s); every pole in the left half plane
```

What the worked example means: The DC gain is 5 and the final value of the step response is 5.

Parallel connection of two transfer functions, `ctrl_parallel_connection`

The transfer function G1 + G2 of two blocks in parallel: (N1 D2 + N2 D1)/(D1 D2).

$$G_1 + G_2 = \frac{N_1 D_2 + N_2 D_1}{D_1 D_2}$$

symbol		meaning	SI unit
`num1`	in	numerator of G1	,
`den1`	in	denominator of G1	,
`num2`	in	numerator of G2	,
`den2`	in	denominator of G2	,
`num`	out	numerator of G1 + G2	,
`den`	out	denominator of G1 + G2	,

```
$ control-systems solve ctrl_parallel_connection "num1=[1]" "den1=[1, 1]" "num2=[2]" "den2=[1,
3]"
num = [3.0,5.0]
den = [1.0,4.0,3.0]
  by: Parallel connection of two transfer functions  (N = N1 D2 + N2 D1, D = D1 D2)
```

What the worked example means: This calculation determines the equivalent transfer function for two system blocks connected in parallel. By taking the first block's numerator [1] and denominator [1, 1], along with the second block's numerator [2] and denominator [1, 3], the algorithm computes a common denominator by multiplying the individual denominators. The new numerator is formed by cross-multiplying each numerator with the other's denominator and summing the results. The final output represents the total gain as a single rational expression, where the resulting numerator is [3, 5] and the denominator is [1, 4, 3].

Poles, zeros and gain of a transfer function, `ctrl_poles_zeros`

The zeros (roots of the numerator), the poles (roots of the denominator) and the zero-pole-gain constant k of G(s) = num/den.

$$G(s) = k \frac{\prod(s - z_i)}{\prod(s - p_j)}$$

symbol		meaning	SI unit
`num`	in	numerator coefficients, highest power of s first	,
`den`	in	denominator coefficients, highest power of s first	,
`zeros`	out	the zeros	,
`poles`	out	the poles	,
`gain`	out	gain k of the zero-pole-gain form (num[0]/den[0])	,

```
$ control-systems solve ctrl_poles_zeros "num=[1, 2]" "den=[1, 4, 3]"
zeros = [-2.0]
poles = [-3.0,-1.0]
gain = 1
  by: Poles, zeros and gain of a transfer function  (roots of both polynomials (Aberth-Ehrlich
iteration))
```

What the worked example means: This calculation finds the roots of the numerator and denominator polynomials to determine the system's zeros and poles. The gain is derived from the ratio of the leading coefficients. For this specific case, the zeros are located at [-2] and the poles at [-3, -1], with a resulting gain value of 1. These values define the dynamic behavior and stability characteristics of the control system.

Series (cascade) connection of two transfer functions, `ctrl_series_connection`

The transfer function G1 G2 of two blocks in series: numerators multiplied, denominators multiplied (no cancellation).

$$G(s) = G_1(s)G_2(s) = \frac{N_1 N_2}{D_1 D_2}$$

symbol		meaning	SI unit
`num1`	in	numerator of G1	,
`den1`	in	denominator of G1	,
`num2`	in	numerator of G2	,
`den2`	in	denominator of G2	,
`num`	out	numerator of G1 G2	,
`den`	out	denominator of G1 G2	,

```
$ control-systems solve ctrl_series_connection "num1=[1]" "den1=[1, 1]" "num2=[2]" "den2=[1,
3]"
num = [2.0]
den = [1.0,4.0,3.0]
  by: Series (cascade) connection of two transfer functions  (polynomial multiplication)
```

What the worked example means: The overall behavior of two blocks connected in series is the product of their individual transfer functions. The resulting numerator is the product of the first numerator [1] and the second numerator [2], giving [2]. Similarly, the resulting denominator is the product of the first denominator [1, 1] and the second denominator [1, 3], giving [1, 4, 3]. This represents the total gain and dynamics of the cascade.

Value of a transfer function at a complex point, `ctrl_tf_evaluate`

G(s0) = num(s0)/den(s0) at s0 = s_re + j s_im: real and imaginary parts, magnitude and angle.

$$G(s_0) = \frac{N(s_0)}{D(s_0)}$$

symbol		meaning	SI unit
`num`	in	numerator coefficients	,
`den`	in	denominator coefficients	,
`s_re`	in	real part of s0	,
`s_im`	in	imaginary part of s0	,
`G_re`	out	real part of G(s0)	,
`G_im`	out	imaginary part of G(s0)	,
`magnitude`	out	|G(s0)|	,
`angle_deg`	out	angle of G(s0) in degrees, (-180, 180]	,

Applies when: s0 is not a pole.

```
$ control-systems solve ctrl_tf_evaluate "num=[1]" "den=[1, 1]" s_re=0 s_im=1
G_re = 0.5
G_im = -0.5
magnitude = 0.7071067812
angle_deg = -45
  by: Value of a transfer function at a complex point  (Horner evaluation in complex
arithmetic)
  s_re = 0
  s_im = 1
  assumes: s0 is not a pole
```

What the worked example means: This calculation determines the value of the transfer function at a specific complex frequency by evaluating the numerator and denominator polynomials. The inputs are the polynomial coefficients for the numerator and denominator, along with the real and imaginary components of the complex variable. The result is expressed as a complex number with a real part of 0.5 and an imaginary part of -0.5, which can also be represented by its magnitude of 0.707107 and phase angle of -45. This value indicates how the system responds to a sinusoidal input at that particular frequency.

Questions and reports: support@ai999x.com. AI999X, The Future Is Self-Improving.